15-288 Ml in a Nutshell

Mohamed Waiel Shikfa

First Edition

Disclaimer

This document aims to summarize the content of the slides and notebooks of 15-288, including what the author considers important. As always, the definition of important is highly subjective so the author might have omitted something that was important to another person, or included something that is trivial to another.

Good luck,

MWS

Contents

Introduction to Machine Learning **2**

ML Tasks: Supervised, Unsupervised, and Reinforcement Learning **5**

The Supervised Learning Workflow and the Canonical SL Problem **9**

Generalization Error and Loss Functions **12**

Regression Workflow, Overfitting, and Cross-Validation **16**

Ordinary Least Squares Regression (OLSR) **20**

k-Nearest Neighbors and Classification Metrics **24**

Outliers: Detection and Handling **28**

Feature Scaling **32**

Correlations, Polynomial Features, and Types of Feature Data **35**

Feature Engineering for Images (Part 1) **40**

Feature Engineering for Images (Part 2) **45**

Feature Engineering for Images (Part 3) **49**

Linear Models: Definitions, Properties, and Training **54**

Gradient Methods for Model Optimization **58**

Support Vector Machines (SVM) **63**

Logistic Regression **67**

Decision Trees **71**

Neural Networks 1: Multi-Layer Perceptron (MLP) **75**

Neural Networks 2: MLPs, Design and Optimization **77**

Neural Networks 3: Convolutional Neural Networks (CNN) **79**

Natural Language Processing, Transformers, and GenAI **81**

Practical Applications: Transformers, LLMs, and GANs **83**

Introduction to Machine Learning

What is Machine Learning?

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) concerned with algorithms that improve automatically through experience. The canonical definition comes from Tom Mitchell (1997):

Definition 1 (Machine Learning (Mitchell, 1997))**.** A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Remark 1 (ML $\subseteq$ AI)**.** ML sits within the broader AI landscape. While classical AI relies on hand-crafted rules and expert systems, ML extracts rules from data. Deep Learning (DL) is in turn a subset of ML that uses multi-layer neural networks to learn hierarchical representations.

In traditional (human) model-based science, a domain expert writes a program that maps inputs to outputs. In ML, the process is inverted: the system is given data and desired outputs, and it *learns* the program (model) that produces the mapping. The learned model can then serve multiple purposes: **making predictions**, **discovering patterns**, or **taking decisions**.

The General ML Scheme

The fundamental setup of any ML problem can be stated concisely:

Definition 2 (Learning Problem)**.** Given a problem domain described by a feature set X and a target set Y, the goal is to learn a mapping:

$$f : X \to Y$$

from a set of training examples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, such that f approximates the true (unknown) mapping F with small error on **unseen** data.

ML Design requires choosing the right *features* (the description language for the data) and the right *model* (the hypothesis class for f) to achieve the task according to the desired performance criterion.

Common instantiations of this scheme include:

Classification: Y is a discrete set of class labels (e.g., spam vs. not spam).

Regression: $Y \subseteq \mathbb{R}$ is a continuous value (e.g., stock price prediction).

Clustering: No target Y is given; the goal is to discover structure in X.

Inductive Learning and Inductive Biases

ML is fundamentally an exercise in **inductive learning**: observing specific examples and generalizing to a broader hypothesis about the world. This contrasts with *deductive learning*, which derives specific conclusions from established logical rules and facts.

Remark 2 (Why Biases Are Necessary)**.** Learning from data is an **ill-posed problem**. Training data is never sufficient to determine a unique solution—multiple hypotheses can be consistent with the same observations. Without additional assumptions, no learner can generalize beyond the training set.

Inductive biases are the set of assumptions a learning algorithm uses to select among hypotheses that are equally consistent with the data. Every ML algorithm embeds some form of inductive bias, whether explicitly (e.g., restricting the hypothesis space to linear functions) or implicitly (e.g., preferring smoother solutions).

Definition 3 (Occam's Razor (Inductive Bias))**.** Among hypotheses consistent with the training data, prefer the **simplest** one. This principle, dating to the 13th century, remains one of the most powerful inductive biases in ML—it underpins regularization, model selection, and the entire bias-variance tradeoff.

Generalization: The Core Challenge

The central goal in ML is **not** to memorize the training data, but to **generalize**—to perform well on future, unseen test data.

Definition 4 (Generalization)**.** The ability of a learned model to achieve good performance on data it has never seen during training. A model that performs well on training data but poorly on test data has **overfit**; it has memorized noise rather than learning the underlying pattern.

Remark 3. Training data are *only* for learning the model. Good performance on training data does **not** guarantee good performance on future data. The core ML challenge is designing algorithms, architectures, and training procedures that ensure generalization.

The ML Pipeline in Production

In practice, the ML algorithm itself is only a small fraction of a production ML system. The full pipeline is an iterative cycle, not a one-shot process:

1. **Problem Definition:** Define objectives, success metrics, and performance criteria.
2. **Data Collection & Management:** Gather relevant operational data from various sources.
3. **Data Wrangling:** Clean, filter, transform, and scale the raw data.
4. **Exploratory Data Analysis (EDA):** Understand distributions, correlations, and anomalies.
5. **Feature Engineering:** Select, extract, and process features; reduce dimensionality.
6. **Model Training:** Choose and train appropriate ML algorithms.
7. **Evaluation & Validation:** Test generalization via cross-validation and held-out test sets.
8. **Deployment & Monitoring:** Deploy the model, monitor performance, and retrain as needed.

Remark 4. The ML code/algorithm typically accounts for a small portion of the overall effort. Data collection, cleaning, feature engineering, and monitoring dominate real-world ML projects.

Main ML Paradigms

Supervised Learning (SL): The training data consists of input-output pairs $\{(\mathbf{x}^{(i)}, y^{(i)})\}$. The model learns a mapping $f : X \to Y$ from labeled examples. Includes *classification* (discrete Y) and *regression* (continuous Y). Core algorithms: k-NN, linear regression, decision trees, SVMs, neural networks.

Unsupervised Learning (UL): The training data contains only inputs $\{\mathbf{x}^{(i)}\}$ with no labels. The goal is to discover hidden structure—clusters, principal components, latent representations. Core algorithms: k-means, PCA, autoencoders.

Reinforcement Learning (RL): An agent interacts with an environment, receiving rewards or penalties for actions. The goal is to learn a policy that maximizes cumulative reward. Not covered in depth in 15-288.

Software Tools Ecosystem

The Python ecosystem is the standard platform for ML and data science. The core stack:

`numpy`: Efficient numerical computation on n-dimensional arrays.

`pandas`: Tabular data manipulation via DataFrames—loading, filtering, grouping.

`matplotlib` / `seaborn`: Data visualization (plots, histograms, heatmaps).

`scikit-learn`: The workhorse ML library—classification, regression, clustering, preprocessing, model selection, and evaluation in a unified API.

Data Exploration with pandas and numpy

```
import pandas as pd
import numpy as np

df = pd.read_csv("data.csv")        # Load tabular data
print(df.shape)                      # (num_rows, num_cols)
print(df.describe())                 # Summary statistics per column
print(df.isnull().sum())             # Count missing values

# Numpy: quick array operations
X = df.drop("target", axis=1).values  # Feature matrix (n x d)
y = df["target"].values               # Target vector (n,)
print(f"Mean: {np.mean(X, axis=0)}")   # Per-feature means
```

End-to-End ML Example with scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load data
X, y = load_iris(return_X_y=True)

# 2. Split into training (80%) and test (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train a k-NN classifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)         # Learn from training data

# 4. Predict on unseen test data
y_pred = model.predict(X_test)

# 5. Evaluate generalization performance
acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {acc:.3f}")   # ~0.967
```

Remark 5. The `train_test_split` step is critical: evaluating on the same data used for training tells you nothing about generalization. Always hold out data the model has never seen.

ML Tasks: Supervised, Unsupervised, and Reinforcement Learning

Machine learning algorithms are organized by the kind of feedback available during training. The three fundamental paradigms—**supervised learning (SL)**, **unsupervised learning (UL)**, and **reinforcement learning (RL)**—differ in what data the learner receives, what signal drives improvement, and what the learned model f ultimately represents.

The General ML Scheme

Every ML system learns a mapping from an input (feature) space to an output (prediction) space:

Definition 5 (Prediction Function)**.** Given a feature space X and an output space Y, the goal of ML is to learn a function $f : X \to Y$ from example data such that f approximates the true (unknown) mapping F with small error on unseen inputs.

The design choices that define a particular ML system are:

- **Hypothesis space** $\mathcal{H}$: the family of candidate functions (e.g., linear models, decision trees, neural networks).
- **Loss / reward signal**: errors from labels (SL), similarity measures (UL), or rewards from an environment (RL).
- **Data**: labeled, unlabeled, or generated through interaction.

Supervised Learning

In supervised learning, a **teacher** (supervisor) provides the learner with labeled training examples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$. The label $y^{(i)}$ tells the learner the correct output for input $\mathbf{x}^{(i)}$. Training proceeds by comparing predictions $f(\mathbf{x}^{(i)})$ to true labels $y^{(i)}$, quantifying the error via a **loss function** ℓ, and adjusting the model to reduce that error.

Remark 6 (Error-Driven Learning)**.** The label provides a precise error signal: the learner knows exactly *how wrong* each prediction is. This is what makes SL the most data-efficient paradigm when labels are available.

Classification

Classification assigns a discrete label to each input.

Definition 6 (Classification)**.** Given a discrete label set $Y = \{c_1, c_2, \ldots, c_n\}$ and a feature space X of dimension k, learn a function $f : X \to Y$ that predicts which class $y \in Y$ a new input $\mathbf{x} \in X$ belongs to.

Binary classification ($|Y| = 2$): $y \in \{0, 1\}$ or $\{-1, +1\}$. Examples: spam vs. not spam, cat vs. dog, healthy vs. sick.

Multi-class classification ($|Y| > 2$): $y \in \{c_1, \ldots, c_n\}$. Examples: weather type (cloudy, snowy, clear), animal species, document topic.

Geometric view. Classification learns a **decision boundary** (separator) in the feature space that partitions regions by class. The separator can be linear (a hyperplane) or arbitrarily non-linear. At test time, a new input $\mathbf{x}^*$ is classified by which region it falls into.

```python
# Classification: Iris dataset with a Decision Tree
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)             # 150 samples, 4 features, 3 classes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
clf = DecisionTreeClassifier(max_depth=3)
clf.fit(X_train, y_train)                     # Learn decision boundary
```

```python
y_pred = clf.predict(X_test)                    # Classify new samples
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

Regression

Regression predicts a continuous value.

Definition 7 (Regression)**.** Given a continuous output space $Y \subseteq \mathbb{R}^m$ (typically $m = 1$) and a feature space X of dimension k, learn $f : X \to Y$ that predicts the response $\mathbf{y} \in Y$ for a new input $\mathbf{x} \in X$.

Univariate regression: One predictor variable. Example: predict electricity usage from temperature alone.

Multivariate regression: Multiple predictor variables. Example: predict electricity usage from temperature, time of day, and day of week.

Geometric view. Regression learns a **curve or surface** in the joint feature–output space that best fits the observed data points. At test time, a new $\mathbf{x}^*$ is mapped onto this curve to produce a predicted $\hat{y}$.

Real-world examples include: house price prediction from (area, rooms, neighborhood), dose–response relationships in medicine, and advertising spend vs. sales revenue.

The SL Workflow

A typical supervised learning pipeline follows these steps:

1. **Collect data:** Gather raw data representative of the problem domain. The dataset should cover all relevant cases proportionally to their real-world frequency.
2. **Label data:** A supervisor annotates each example with the correct output y.
3. **Extract features:** Transform raw data (images, text, signals) into a numerical feature vector $\mathbf{x} \in \mathbb{R}^k$.
4. **Train model:** Select a hypothesis space $\mathcal{H}$, a loss function ℓ, and an optimization method. Minimize empirical risk over the training set.
5. **Predict:** Apply the trained model f to new, unseen inputs.

The Cost of Labels

Supervised learning is powerful but expensive. Data labeling is:

- **Time-consuming and costly:** Human annotators must inspect every example—thousands or millions of them.
- **Error-prone:** Image labeling requires subjective judgment (is that a "cat" or a "cat with a mask"?). Handwritten digit recognition (OCR) introduces transcription errors. Text annotation (document classification, entity tagging) is lengthy and inconsistent across annotators.
- **Domain-dependent:** Medical imaging requires expert radiologists; legal documents require trained lawyers.

These costs motivate unsupervised and self-supervised approaches, including modern LLMs that use self-supervised pre-training (and optionally RLHF fine-tuning) to reduce dependence on manual labels.

Unsupervised Learning

In unsupervised learning, the training data consists of inputs *only*—no labels are provided. The goal is to discover **hidden structure**, patterns, or relationships within the data.

Remark 7 (No Error Signal)**.** Without labels, there is no "correct answer" to compare against. Instead, UL algorithms rely on **similarity measures** and structural assumptions (inductive biases) to organize the data.

Clustering

Clustering groups data items so that items within the same cluster are more similar to each other than to items in other clusters.

Definition 8 (Clustering)**.** Given N data points $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$ in a k-dimensional feature space $\mathcal{F}^k$ and a similarity/dissimilarity measure, partition the data into n clusters. Formally, learn $f : \mathcal{F}^k \to \{1, 2, \ldots, n\}$.

Key challenges:

- The number of clusters n is usually *not known* in advance and must be estimated.
- The choice of similarity measure is an **inductive bias**: Euclidean distance, cosine similarity, and correlation each yield different clusterings of the same data.
- Ambiguity is inherent—"group these people into clusters" has many valid answers depending on the criteria (age? height? interests?).

```
# Clustering: KMeans on synthetic blobs
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=300, centers=4,   # 4 ground-truth clusters
                  cluster_std=0.8, random_state=42)
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)                                  # Discover cluster structure
labels = kmeans.labels_                        # Cluster assignment per point
centers = kmeans.cluster_centers_              # Centroid of each cluster
print(f"Inertia (within-cluster SSE): {kmeans.inertia_:.1f}")
```

Dimensionality Reduction / Compression

Dimensionality reduction identifies that high-dimensional data often lies on a lower-dimensional manifold. By finding hidden variables or redundant features, we can compress the representation without significant information loss.

Example: if data in $\mathbb{R}^3$ actually lies on a 2D plane, one feature is irrelevant and can be discarded. Principal Component Analysis (PCA) is the classical technique.

Generative Models

Generative models estimate the probability density $p(\mathbf{x})$ of the data—the distribution that generated the observed samples. This captures probabilistic relationships and correlations among features, enabling the model to generate new, plausible data points.

Reinforcement Learning

Reinforcement learning differs fundamentally from both SL and UL: there are no labels *and* no pre-collected dataset.

Definition 9 (Reinforcement Learning)**.** An **agent** interacts with an **environment** by taking actions, observing states, and receiving **rewards** (or costs). The goal is to learn an optimal **policy**—a mapping from states to actions—that maximizes cumulative reward over time.

Remark 8. The agent must generate its own training data through exploration. Rewards may be sparse (received only after many actions) and delayed, making credit assignment difficult. RL addresses **sequential decision-making**: each action affects future states and rewards.

RL is not covered in depth in this course (see 15-281 for a full treatment), but the paradigm is important to recognize: it underlies game-playing AI, robotics, and the RLHF stage used to fine-tune modern LLMs.

Comparison of the Three Paradigms

	Supervised	**Unsupervised**	**Reinforcement**
Data	Labeled	Unlabeled	Not given (agent explores)
Feedback signal	Error from labels	Similarity measures	Rewards / costs
Training data	Given	Given	Generated by agent
Goal	Predict y for new $\mathbf{x}$	Discover structure	Learn optimal policy
Typical tasks	Classification, regression	Clustering, PCA, density est.	Sequential decisions

Remark 9 (Choosing a Paradigm)**.** Having labeled data is a clear advantage—SL provides the strongest learning signal. But acquiring labels is expensive and error-prone, which is why UL and self-supervised methods are increasingly important. RL applies when the problem is inherently sequential and no static dataset can capture the dynamics.

The Supervised Learning Workflow and the Canonical SL Problem

SL Classification Workflow: A Running Example

Consider a concrete task: given data about cars, learn to predict whether a car is a **family car** or not. Each car is described by measurable attributes (price, engine power, color, ...), and labeled + (family car) or − (not family car). The goal is to learn a mapping $f : X \to Y$ that generalizes to unseen cars.

Definition 10 (Supervised Learning Dataset)**.** A dataset D consists of m labeled examples:

$$D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$$

where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is the **feature vector** (the numeric representation of input i) and $y^{(i)} \in \mathcal{Y}$ is the **label** (the target output). For binary classification, $\mathcal{Y} = \{0, 1\}$ (or $\{-1, +1\}$).

In the car example, each car i is represented by a two-dimensional feature vector:

$$\mathbf{x}^{(i)} = \begin{bmatrix} \text{price}^{(i)} \\ \text{engine_power}^{(i)} \end{bmatrix} \in \mathbb{R}^2, \qquad y^{(i)} = \begin{cases} 1 & \text{if } \mathbf{x}^{(i)} \text{ is a family car} \\ 0 & \text{otherwise} \end{cases}$$

The function we seek maps features to categories: $f : (\text{price}, \text{power}) \to \{0, 1\}$, or more generally, $f : \mathbb{R}^d \to \mathcal{Y}$.

Exploratory Data Analysis (EDA)

Before modeling, **Exploratory Data Analysis (EDA)** examines the dataset to inform design choices:

- Visualize examples in the feature space (e.g., scatter plot of price vs. engine power, colored by class).
- Compute summary statistics: class balance, feature ranges, medians per class, correlations.
- Identify potential issues: missing values, outliers, class imbalance.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification

# Generate a toy 2-feature binary classification dataset
X, y = make_classification(n_samples=50, n_features=2,
    n_redundant=0, n_clusters_per_class=1, random_state=42)

# EDA: scatter plot in feature space
plt.scatter(X[y==0, 0], X[y==0, 1], marker='o', label='Class 0')
plt.scatter(X[y==1, 0], X[y==1, 1], marker='+', label='Class 1')
plt.xlabel('Feature 1 (e.g., price)')
plt.ylabel('Feature 2 (e.g., engine power)')
plt.legend(); plt.title('Dataset in Feature Space'); plt.show()
```

The Hypothesis Class $\mathcal{H}$

The core question is: *what is the relationship f between the features and the target?* We do not know the true f, so we must choose a **hypothesis class** $\mathcal{H}$—a family of candidate functions—and search within it for a good approximation $\hat{f} \in \mathcal{H}$.

Definition 11 (Hypothesis Class and Inductive Bias)**.** The **hypothesis class** $\mathcal{H}$ is the set of all functions we consider as candidates for the true mapping f. The act of choosing $\mathcal{H}$ constitutes our **inductive bias**: we commit to explaining the data through the lens of $\mathcal{H}$, which constrains what patterns we can express.

Remark 10 (Examples of Hypothesis Classes)**.** For a 2D classification problem, $\mathcal{H}$ could be:

Axis-aligned rectangles: Decision regions of the form $(p_1 \leq x_1 \leq p_2) \wedge (e_1 \leq x_2 \leq e_2)$. Four parameters.

Circles: $(x_1 - c_1)^2 + (x_2 - c_2)^2 - r^2 = 0$. Three parameters.

Ellipses: $\frac{(x_1-c_1)^2}{a^2} + \frac{(x_2-c_2)^2}{b^2} - 1 = 0$. Four or five parameters (with rotation).

Linear separators: $w_1 x_1 + w_2 x_2 + w_0 = 0$. Three parameters.

Each choice imposes a different geometric structure on the decision boundary.

In practice, you rarely define $\mathcal{H}$ analytically. Different ML **algorithms** (SVM, decision trees, neural networks) implicitly define different hypothesis spaces through their architecture and parameters. The key insight: every algorithm carries an inductive bias that determines what it can and cannot learn.

Parametric Hypotheses

Most hypothesis classes are **parametric**: the candidate function is fully determined by a parameter vector $\boldsymbol{\theta}$.

Definition 12 (Parametric Hypothesis)**.** A parametric hypothesis $f_{\boldsymbol{\theta}}(\mathbf{x})$ is a function whose behavior is entirely determined by a finite-dimensional parameter vector $\boldsymbol{\theta} \in \mathbb{R}^p$. Learning reduces to finding the optimal $\boldsymbol{\theta}^*$.

For axis-aligned rectangles: $\boldsymbol{\theta} = (p_1, p_2, e_1, e_2)$ and $f_{\boldsymbol{\theta}}(\mathbf{x}) = 1$ iff $\mathbf{x}$ falls inside the rectangle. For a linear classifier: $\boldsymbol{\theta} = (w_0, w_1, w_2)$ and $f_{\boldsymbol{\theta}}(\mathbf{x}) = \text{sgn}(w_0 + w_1 x_1 + w_2 x_2)$.

```
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

# Different algorithms = different implicit hypothesis spaces
svm_model = SVC(kernel='linear')   # H = linear separators
tree_model = DecisionTreeClassifier(max_depth=3)  # H = axis-aligned splits

svm_model.fit(X, y)   # Find best linear boundary
tree_model.fit(X, y)   # Find best tree partition

# Each model's decision boundary has a fundamentally different shape
print(f"SVM accuracy:  {svm_model.score(X, y):.2f}")
print(f"Tree accuracy: {tree_model.score(X, y):.2f}")
```

Consistent Hypotheses

One natural criterion: find $\hat{f} \in \mathcal{H}$ that makes *zero* mistakes on the training data.

Definition 13 (Consistent Hypothesis)**.** A hypothesis $\hat{f}$ is **consistent** with dataset D if it correctly classifies every training example:

$$\hat{f}(\mathbf{x}^{(i)}) = y^{(i)} \quad \forall\, (\mathbf{x}^{(i)}, y^{(i)}) \in D$$

There may be many consistent hypotheses within $\mathcal{H}$. Two extremes are informative:

Most specific (S): The tightest boundary that still contains all positive examples. Minimal generalization beyond the data.

Most general (G): The broadest boundary consistent with the data. Maximal generalization given the constraint of zero training error.

Remark 11 (Consistency $\neq$ Generalization)**.** Three critical caveats about consistency:

1. A consistent hypothesis can still be **wrong on unseen data**—it merely memorized the training set without capturing the true pattern.
2. Consistency may be **impossible** given a particular $\mathcal{H}$ (e.g., linearly inseparable data with a linear hypothesis class).
3. For small datasets, consistency is especially unreliable as a proxy for true performance.

The Loss Function

Since consistency alone is insufficient, we need a principled way to measure *how wrong* a hypothesis is. This is the role of the **loss function**.

Definition 14 (Loss Function)**.** A **loss function** $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ maps a (prediction, true label) pair to a non-negative real number quantifying the error:

$$\ell(f_{\boldsymbol{\theta}}(\mathbf{x}),\ y) \to \mathbb{R}$$

Smaller loss means better prediction. The loss is zero when the prediction is perfect.

Definition 15 (0/1 Loss for Classification)**.** For binary classification, the simplest loss counts misclassifications:

$$\ell_{0/1}\Big(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}),\ y^{(i)}\Big) = \begin{cases} 1 & \text{if } f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \neq y^{(i)} \\ 0 & \text{if } f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = y^{(i)} \end{cases}$$

Summing over the dataset gives the total number of errors. A consistent hypothesis achieves $\sum \ell_{0/1} = 0$.

The loss on training data estimates the *risk* of making future mistakes: a hypothesis with loss 3 on the training set is presumably worse than one with loss 1, though neither guarantee is absolute for unseen data.

```
import numpy as np

# Suppose we have predictions and true labels
y_true = np.array([1, 0, 1, 1, 0, 0, 1, 0, 1, 1])
y_pred = np.array([1, 0, 0, 1, 0, 1, 1, 0, 1, 0])

# Compute 0/1 loss manually
zero_one_losses = (y_pred != y_true).astype(int)  # per-example loss
total_loss = zero_one_losses.sum()                 # total misclassifications
empirical_risk = zero_one_losses.mean()            # average loss (ERM objective)

print(f"Per-example losses: {zero_one_losses}")    # [0 0 1 0 0 1 0 0 0 1]
print(f"Total errors: {total_loss}")               # 3
print(f"Empirical risk: {empirical_risk:.2f}")     # 0.30
```

The Canonical Supervised Learning Problem

We can now state the central optimization problem that unifies all supervised learning.

Definition 16 (Empirical Risk Minimization (ERM))**.** Given a training set $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, a parametric hypothesis $f_{\boldsymbol{\theta}}$, and a loss function ℓ, the **Canonical SL Problem** is to find the parameter vector $\boldsymbol{\theta}^*$ that minimizes the **empirical risk**:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}}\ L_{\text{emp}}(\boldsymbol{\theta}; D) = \arg\min_{\boldsymbol{\theta}}\ \frac{1}{m}\sum_{i=1}^{m} \ell\Big(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}),\ y^{(i)}\Big)$$

This is called **Empirical Risk Minimization (ERM)**: we minimize the sample average of losses over the training data.

Every supervised learning algorithm—linear regression, logistic regression, SVMs, neural networks—is an instantiation of this template. They differ in three design choices:

1. **Hypothesis class** $\mathcal{H}$: What form does $f_{\boldsymbol{\theta}}$ take?
2. **Loss function** ℓ: How do we penalize wrong predictions?
3. **Optimization method**: How do we solve the arg min?

Remark 12 (ERM Is Necessary but Not Sufficient)**.** Minimizing training error is a *necessary* condition for a good model, but it is **not sufficient**. A model that perfectly fits the training data (zero empirical risk) may perform terribly on unseen data—this is **overfitting**. The true goal is to minimize the *generalization error* (the expected loss on new, unseen examples drawn from the same distribution). Bridging the gap between empirical risk and true risk is the central challenge of statistical learning theory, addressed via regularization, validation, and capacity control.

Generalization Error and Loss Functions

Empirical Risk vs. Generalization Error

Training a model means minimizing the loss on the data we *have*. But the real objective is to minimize the loss on data we have *never seen*. This gap—between training performance and true performance—is the central tension in all of supervised learning.

Definition 17 (Empirical Risk)**.** Given a training set $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$ and a loss function ℓ, the **empirical risk** (or empirical error) is the average loss over the training examples:

$$L_{\text{emp}}(\boldsymbol{\theta}; D) = \frac{1}{m} \sum_{i=1}^{m} \ell\Big(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}),\ y^{(i)}\Big)$$

This is the quantity we directly minimize during training.

Definition 18 (Generalization Error)**.** The **generalization error** (also called out-of-sample error or risk) is the expected loss over the entire problem domain:

$$L_{\text{gen}} = \sum_{i=1}^{M} \ell\Big(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}),\ y^{(i)}\Big) \cdot P(\mathbf{x}^{(i)}, y^{(i)})$$

where $P(\mathbf{x}, y)$ is the (unknown) prior probability distribution over all M possible input–output pairs in the problem domain, and the sum runs over every feasible pair.

Remark 13 (We Can Never Compute L_{gen} Exactly)**.** Computing L_{gen} requires (1) knowing every possible data pair $(\mathbf{x}, y)$ the model could encounter, and (2) knowing their joint probability $P(\mathbf{x}, y)$. Both are unknown and unknowable in practice. We can only **estimate** generalization error:

- **Theoretically:** derive statistical bounds on $|L_{\text{gen}} - L_{\text{emp}}|$ (PAC learning, VC dimension).
- **Empirically:** measure L_{emp} on held-out validation/test data as a statistical estimate of L_{gen}.

Remark 14 (Training Loss Is a Local Indicator)**.** L_{emp} on the training set tells you how well the model fits the data it was trained on—nothing more. A model can drive training loss to zero by memorizing the dataset (overfitting) while performing terribly on unseen data. The empirical loss on a **validation set** (data not used during training) is what actually estimates L_{gen}.

Promoting Generalization

Since we cannot compute L_{gen} directly, we use several strategies to ensure that low L_{emp} translates into low L_{gen}:

Act on the loss function ℓ: Add **regularization** terms $\lambda R(\mathbf{w})$ to penalize model complexity (e.g., L_1, L_2 penalties on weights). This discourages overfitting by preventing weights from growing too large.

Act on model complexity: Choose a hypothesis class $\mathcal{H}$ whose capacity matches the problem. Too flexible $\Rightarrow$ overfitting; too rigid $\Rightarrow$ underfitting.

Act on the data: Gather large, representative datasets. As $m \to M$, the training distribution approaches the true distribution, and $L_{\text{emp}} \to L_{\text{gen}}$.

Train/Validation/Test splits: Partition data to separately optimize parameters (train), select model/hyperparameters (validation), and estimate final generalization (test).

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

# Generate synthetic data
np.random.seed(42)
```

```python
X = np.random.randn(200, 5)
y = X @ np.array([3, -1, 0.5, 0, 0]) + np.random.randn(200) * 0.5

# Split into train / validation / test
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5)

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)

train_err = mean_squared_error(y_train, model.predict(X_train))
val_err   = mean_squared_error(y_val,   model.predict(X_val))
test_err  = mean_squared_error(y_test,  model.predict(X_test))

print(f"Train MSE: {train_err:.4f}")  # L_emp on training set
print(f"Val   MSE: {val_err:.4f}")    # Estimate of L_gen
print(f"Test  MSE: {test_err:.4f}")   # Final L_gen estimate
# Gap between train and val error = generalization gap
```

Loss Functions for Classification

In binary classification with labels $y \in \{-1, +1\}$, every classification loss can be expressed as a function of the **margin**.

Definition 19 (Margin)**.** The **margin** of a prediction is defined as:

$$\mu(\mathbf{x}) = f_{\mathbf{w}}(\mathbf{x}) \cdot y(\mathbf{x})$$

- $\mu > 0$: the prediction **agrees** with the true label (correct).
- $\mu < 0$: the prediction **disagrees** with the true label (incorrect).
- $|\mu|$: the **confidence** of the prediction.

The margin converts the two-argument problem $\ell(\hat{y}, y)$ into a single-argument function $\ell(\mu)$, making it easier to analyze and compare losses.

Classification Loss Functions

All of the following are functions of the margin μ:

0/1 Loss: $\ell(\mu) = \begin{cases} 1 & \text{if } \mu < 0 \\ 0 & \text{otherwise} \end{cases}$

The ideal loss—it counts misclassifications exactly. However, it is a step function: non-convex, non-differentiable, and computationally intractable to optimize. All other classification losses are differentiable **surrogate** approximations of 0/1 loss.

Hinge Loss: $\ell(\mu) = \max(0,\ 1 - \mu)$

Used in **Support Vector Machines**. Zero loss only when $\mu \geq 1$ (i.e., correct *and* confident). Penalizes predictions that are correct but too close to the boundary. Convex but not differentiable at $\mu = 1$.

Logistic Loss: $\ell(\mu) = \log(1 + e^{-\mu})$

Used in **Logistic Regression**. Smooth, differentiable everywhere, and convex—ideal for gradient-based optimization. Never reaches exactly zero: even highly confident correct predictions incur a small loss, continuously pushing the model to improve.

Exponential Loss: $\ell(\mu) = e^{-\mu}$

Used in **AdaBoost**. Extremely sensitive to misclassifications ($\mu \ll 0$ yields enormous loss), making it vulnerable to outliers and label noise.

Remark 15 (Asymmetric Loss)**.** The 0/1 loss treats all misclassifications equally, but in many real problems the costs are asymmetric. Misclassifying a sick patient as healthy (false negative) is far worse than misclassifying a healthy patient as sick (false positive). An **asymmetric loss** assigns different penalties $a \neq b$ to different error types via a cost matrix:

$$A = \begin{bmatrix} 0 & a \\ b & 0 \end{bmatrix}, \quad a \neq b$$

```
import numpy as np

mu = np.linspace(-3, 3, 500)  # Margin values

# Classification losses as functions of margin
zero_one  = (mu < 0).astype(float)              # 0/1 loss
hinge     = np.maximum(0, 1 - mu)               # Hinge loss (SVM)
logistic  = np.log(1 + np.exp(-mu))              # Logistic loss
exponential = np.exp(-mu)                        # Exponential loss (AdaBoost)

# --- Plotting (requires matplotlib) ---
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.plot(mu, zero_one, 'k--', lw=2, label='0/1 Loss')
plt.plot(mu, hinge, 'b',   lw=2, label='Hinge Loss')
plt.plot(mu, logistic, 'r', lw=2, label='Logistic Loss')
plt.plot(mu, exponential, 'g', lw=2, label='Exponential Loss')
plt.xlabel('Margin $\mu$'); plt.ylabel('Loss $\ell(\mu)$')
plt.legend(); plt.title('Classification Loss Functions')
plt.axvline(0, color='gray', ls=':'); plt.ylim(-0.1, 4)
plt.savefig('classification_losses.png', dpi=150)
```

Loss Functions for Regression

For regression tasks where $y \in \mathbb{R}$, the loss is a function of the **residual** $r = y - \hat{y}$, measuring how far the prediction deviates from the true value.

Squared Loss (L_2): $\ell(y, \hat{y}) = (y - \hat{y})^2$

The workhorse of regression. Squaring makes small residuals ($|r| < 1$) even smaller and large residuals ($|r| > 1$) much larger, so the model is heavily influenced by **outliers**. Smooth and differentiable everywhere, yielding clean gradients: $\frac{\partial \ell}{\partial \hat{y}} = -2(y - \hat{y})$.

Absolute Loss (L_1): $\ell(y, \hat{y}) = |y - \hat{y}|$

Linear penalty regardless of error magnitude, making it **robust to outliers**. The gradient is constant in magnitude (± 1), which avoids the exploding-gradient effect of L_2 on large residuals. However, it is non-differentiable at $r = 0$.

Huber Loss: $\ell_\delta(r) = \begin{cases} \frac{1}{2} r^2 & \text{if } |r| \leq \delta \\ \delta\,|r| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$

Combines the best of both: quadratic near zero (smooth gradients for small errors) and linear in the tails (robustness to outliers). The threshold δ controls the transition point.

```
import numpy as np

r = np.linspace(-4, 4, 500)  # Residuals

# Regression losses
l2_loss   = r ** 2                               # Squared loss
l1_loss   = np.abs(r)                            # Absolute loss
delta = 1.0
huber = np.where(np.abs(r) <= delta,             # Huber loss
                 0.5 * r**2,
                 delta * np.abs(r) - 0.5 * delta**2)
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.plot(r, l2_loss, 'r', lw=2, label='$L_2$ (Squared)')
plt.plot(r, l1_loss, 'b', lw=2, label='$L_1$ (Absolute)')
plt.plot(r, huber,   'g', lw=2, label=f'Huber ($\delta$={delta})')
plt.xlabel('Residual $r = y - \hat{y}$')
plt.ylabel('Loss $\ell(r)$')
plt.legend(); plt.title('Regression Loss Functions')
plt.savefig('regression_losses.png', dpi=150)
```

The Supervised Learning Design Framework

Every supervised learning problem requires four design choices. Each interacts with the others—changing one may require revisiting the rest.

1. **Hypothesis class $\mathcal{H}$:** What family of functions can $f_{\boldsymbol{\theta}}$ belong to? (Linear models, decision trees, neural networks, …) This is the **inductive bias**—our structural assumption about the mapping $f : X \to Y$.
2. **Loss function ℓ:** How do we quantify prediction errors? The choice of loss encodes what types of errors matter most and directly shapes the optimization landscape.
3. **Optimization algorithm:** How do we search for $\boldsymbol{\theta}^*$? Closed-form solutions exist only in special cases (e.g., OLS). In general, we use iterative methods (gradient descent, SGD, Adam, …).
4. **Validation procedure:** How do we estimate L_{gen} and select among competing models? (Hold-out split, k-fold cross-validation, …)

Remark 16 (The Design Choices Are Not Independent)**.** The hypothesis class constrains which loss functions are computationally tractable (e.g., hinge loss pairs naturally with SVMs). The loss function's differentiability determines which optimizers can be used. The amount of available data influences how much model complexity we can afford and which validation strategy is feasible.

Regression Workflow, Overfitting, and Cross-Validation

A Regression Example: End-to-End Workflow

To ground the abstract ERM framework in a concrete workflow, consider a canonical regression task. We observe $m = 10$ noisy samples from an unknown function $f(x) = \sin(2\pi x)$, where $x \in [0, 1]$ and $t \in [-1, 1]$. The dataset is:

$$D = \{(x_i, t_i)\}_{i=1}^m, \quad t_i = \sin(2\pi x_i) + \epsilon_i$$

where ϵ_i is random noise. The goal: learn $f : X \to T$ from D alone—without knowing the true generating function.

Step 1: Choose the Hypothesis Class $\mathcal{H}$

We hypothesize that f belongs to the family of **polynomial functions of degree** M:

$$f_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_M x^M = \sum_{j=0}^{M} \theta_j x^j$$

with parameter vector $\boldsymbol{\theta} \in \mathbb{R}^{M+1}$. Two critical observations:

- M controls the **complexity/capacity** of the hypothesis class. Higher M means more expressive models.
- The model is **linear in its parameters** $\boldsymbol{\theta}$ (even though it is non-linear in x), so standard linear algebra solvers apply.
- A degree-M polynomial has $M + 1$ free parameters to learn.

Step 2: Define the Loss Function

Definition 20 (Sum of Squared Errors (SSE))**.**

$$\mathrm{SSE}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{m} (f_{\boldsymbol{\theta}}(x_i) - t_i)^2$$

Definition 21 (Root Mean Square Error (RMSE))**.**

$$\mathrm{RMSE}(\boldsymbol{\theta}) = \sqrt{\frac{\mathrm{SSE}(\boldsymbol{\theta})}{m}}$$

RMSE normalizes by dataset size and has the same units as the target, making it interpretable and comparable across datasets of different sizes. This serves as our empirical loss L_{emp}.

Step 3: Train (Minimize L_{emp})

For each choice of M, we solve $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathrm{SSE}(\boldsymbol{\theta})$ using least-squares. The following snippet fits polynomials of varying degrees to the $\sin(2\pi x)$ dataset:

```
import numpy as np
from numpy.polynomial import polynomial as P

# Generate noisy sin(2*pi*x) data
np.random.seed(0)
x = np.linspace(0, 1, 10)
t = np.sin(2 * np.pi * x) + np.random.randn(10) * 0.3

# Fit polynomials of degree M = 0, 1, 3, 9
for M in [0, 1, 3, 9]:
    coeffs = np.polyfit(x, t, M)    # Least-squares fit
```

```
    p = np.poly1d(coeffs)           # Callable polynomial
    residuals = p(x) - t
    rmse = np.sqrt(np.mean(residuals**2))
    print(f"M={M}: RMSE={rmse:.4f}, params={M+1}")
```

Remark 17 (Parameters vs. Data Points)**.** A degree-M polynomial has $M+1$ parameters. With $m = 10$ data points: $M = 0$ has 1 parameter (constant), $M = 1$ has 2 (line), $M = 3$ has 4 (cubic), and $M = 9$ has 10—exactly as many parameters as data points.

Overfitting vs. Generalization

Training error (L_{emp} on D) tells us how well the model fits the data it has seen. But the real question is: *how will it perform on new, unseen data?*

The Core Phenomenon

As model complexity M increases:

1. **Training RMSE decreases monotonically.** More parameters $\Rightarrow$ more capacity to fit the training points. At $M = 9$ with $m = 10$ points, the polynomial passes through *every* data point: $\text{RMSE}_{\text{train}} = 0$.
2. **Validation RMSE first decreases, then rapidly increases.** The model begins fitting the noise rather than the underlying signal.

Definition 22 (Overfitting)**.** **Overfitting** occurs when a model fits too closely to the training data—learning the noise, not the signal—resulting in excellent training performance but poor generalization to unseen data. Overfitting defeats the fundamental purpose of ML.

Remark 18 (When Does Overfitting Occur?)**.** Overfitting depends on the relationship between **model complexity** and **data quantity**:

- When $M \geq m$ (parameters $\geq$ data points), the model can interpolate every training point perfectly, almost certainly overfitting.
- When $N \gg M$ (much more data than parameters), even complex models can generalize well—there is enough signal to constrain the parameters.
- Simpler models tend to prevent overfitting and generalize better. This principle motivates **regularization** and connects to Occam's razor: prefer the simplest hypothesis consistent with the data.

The Sweet Spot

The optimal model complexity corresponds to the point where:

- Validation error is near its **minimum**, and
- Training error is reasonably low (but *not* zero—a zero training error on noisy data is a red flag).

Model Selection: The Bias-Variance Tradeoff

The tension between underfitting and overfitting is formalized by the **bias-variance tradeoff**:

Bias: Error from overly simplistic assumptions in the hypothesis class. High bias $\Rightarrow$ the model systematically misses the true pattern (**underfitting**).

Variance: Error from excessive sensitivity to the specific training set. High variance $\Rightarrow$ the model changes drastically with different samples (**overfitting**).

Increasing complexity *reduces* bias but *increases* variance. The goal of model selection is to find the complexity that minimizes the **total** generalization error, which is approximately $\text{Bias}^2 + \text{Variance} + \text{irreducible noise}$.

The Hold-Out Method

The simplest approach to estimating generalization error: split the available data into two disjoint sets.

Definition 23 (Hold-Out Method)**.** Partition the dataset into:

- **Training set** D: used to learn parameters $\boldsymbol{\theta}^*$.
- **Test set** T: used to evaluate $L_{\text{emp}}(T)$ as an estimate of L_{gen}.

The fundamental rationale: evaluate the model on data **independent** from the data used for training.

Remark 19 (Drawbacks of Hold-Out)**.**

- **Wastes data:** test examples are never used for training, which hurts performance when data is scarce.
- **High variance:** a single (un)fortunate split can produce a misleading estimate of generalization error.
- **Split dependency:** results can vary significantly depending on which examples end up in which set.

Cross-Validation (CV)

Cross-validation overcomes the limitations of hold-out by iterating the split-train-evaluate procedure K times, rotating which data plays the training and validation roles.

K-Fold Cross-Validation

Definition 24 (K-Fold CV)**.** Partition the dataset into K equal-sized **folds** $F_1, F_2, \ldots, F_K$. For each fold $k = 1, \ldots, K$:

1. **Train** on all folds except F_k: $D_{\text{train}} = D \setminus F_k$.
2. **Validate** on F_k: compute $L_{\text{emp}}(F_k)$.

The CV estimate of generalization error is the average across folds:

$$L_{\text{CV}} = \frac{1}{K} \sum_{k=1}^{K} L_{\text{emp}}(F_k)$$

Every data point serves as a validation example exactly once and as a training example $K - 1$ times. Common choice: $K = 5$ or $K = 10$.

Variants

Leave-One-Out CV (LOOCV): The extreme case $K = N$. Each fold contains a single example. Maximizes training data per fold but is computationally expensive (N training runs) and can have high variance for small datasets.

Random Subsampling CV: Instead of deterministic partitions, randomly sample training/validation splits each iteration. More flexible but some points may never appear in validation.

Remark 20 (Formal Guarantees)**.** Under mild conditions and reasonably large datasets, CV provides not just an empirical estimate of generalization error, but comes with formal bounds relating the CV estimate to the true (unknown) L_{gen}.

K-Fold CV with `sklearn`

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score

# Generate noisy sin(2*pi*x) data
np.random.seed(0)
X = np.linspace(0, 1, 30).reshape(-1, 1)
y = np.sin(2 * np.pi * X).ravel() + np.random.randn(30) * 0.3
```

```python
# Compare polynomial degrees via 5-fold CV
for M in [1, 3, 5, 9]:
    model = make_pipeline(
        PolynomialFeatures(M), LinearRegression()
    )
    # neg_mean_squared_error: sklearn negates so higher=better
    scores = cross_val_score(
        model, X, y, cv=5, scoring='neg_mean_squared_error'
    )
    rmse_scores = np.sqrt(-scores)
    print(f"M={M}: CV RMSE = {rmse_scores.mean():.3f}"
          f" (+/- {rmse_scores.std():.3f})")
```

The General Model Selection Pipeline

In practice, model selection combines hold-out and cross-validation in a principled pipeline:

1. **Initial split (Hold-Out):** Partition the full dataset into a **Training set** and a **Test set**. The test set is locked away—untouched until the very end.
2. **Internal CV:** Further split the Training set into internal training folds and validation folds using K-Fold CV.
3. **Train and validate candidates:** For each candidate model (varying hyperparameters, algorithms, or complexity), run the full CV loop. Record each model's estimated generalization error L_{CV}.
4. **Select the best model:** Choose the candidate with the lowest L_{CV}.
5. **Retrain on full Training set:** Retrain the selected model using the *entire* Training set (all folds combined) to maximize the data available for learning.
6. **Final evaluation on Test set:** Evaluate the retrained model on the held-out Test set. This is the unbiased estimate of deployment performance. If the test error is much larger than L_{CV}, reconsider the candidate models or data pipeline.

Remark 21 (Why Retrain?)**.** After CV selects the best model *type* (e.g., $M = 3$), we retrain on all training data because CV only used $\frac{K-1}{K}$ of it per fold. The final model benefits from every available training example.

Remark 22 (Test Set Discipline)**.** The test set must remain untouched during model selection. If you use test performance to choose between models, it becomes a validation set—and your generalization estimate becomes optimistically biased. This is a common and serious methodological error.

Ordinary Least Squares Regression (OLSR)

Linear Regression Setup

We hypothesize that the unknown target function $F(\mathbf{x})$ is approximately linear: the prediction $\hat{y}$ is a weighted sum of input features.

Definition 25 (Linear Regression Model)**.** Given an input $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T \in \mathbb{R}^d$, the linear model predicts:

$$\hat{y}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = \mathbf{w}^T \tilde{\mathbf{x}}$$

where $\mathbf{w} = [w_0, w_1, \ldots, w_d]^T$ is the **weight vector** (including bias w_0) and $\tilde{\mathbf{x}} = [1, x_1, \ldots, x_d]^T$ is the **augmented feature vector** with the bias absorbed.

Each weight w_i quantifies the contribution of feature x_i to the prediction. The bias w_0 is a constant offset independent of any feature value.

Remark 23 (Linearity is in the Weights)**.** During training, the features x_i are known constants (data) while the weights w_i are the variables to be optimized. The model is **linear in the parameters $\mathbf{w}$**—this is what matters for deriving the closed-form solution, not whether the original features are transformed non-linearly.

Regression Terminology

Simple regression: One predictor, one response: $y = f(x)$.

Univariate regression: One predictor, one (or more) response variables.

Multiple regression: Multiple predictors, one response: $y = f(\mathbf{x})$, where $f : \mathbb{R}^d \to \mathbb{R}$.

Multivariate regression: One or more predictors, multiple response variables: $\mathbf{y} = f(\mathbf{x})$.

Our focus is **multiple regression**: d predictors mapping to a single scalar output.

The Design Matrix

Given a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ with N examples and d features, we stack all augmented inputs into the **design matrix** and all targets into a vector:

$$\tilde{\mathbf{X}} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & \cdots & x_d^{(N)} \end{bmatrix}_{N \times (d+1)}, \qquad \mathbf{Y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}_{N \times 1}$$

The vector of all predictions is then a single matrix–vector product: $\hat{\mathbf{Y}} = \tilde{\mathbf{X}}\mathbf{w}$.

Why Linear Models?

- **Simplicity:** Easy to implement, interpret, and debug. Each weight directly indicates a feature's contribution.
- **Efficiency:** Predictions reduce to a dot product—trivially parallelizable on GPUs.
- **Extensibility:** Non-linear feature transforms $\phi(\mathbf{x})$ let the model capture non-linear patterns while remaining linear in the parameter space.
- **Foundation:** Linear models underpin neural networks, SVMs, and many ensemble methods.

Loss Function: Mean Squared Error

Following the ERM framework, we choose the **Squared Error** loss and average over all N training examples.

Definition 26 (MSE Loss for Linear Regression)**.**

$$\ell(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \mathbf{w}^T\tilde{\mathbf{x}}^{(i)}\right)^2$$

In compact matrix form:

$$\ell(\mathbf{w}) = \frac{1}{N}\left(\mathbf{Y} - \tilde{\mathbf{X}}\mathbf{w}\right)^T\left(\mathbf{Y} - \tilde{\mathbf{X}}\mathbf{w}\right)$$

This is equivalent to the **Sum of Squared Errors (SSE)** scaled by $\frac{1}{N}$. When this squared-error loss is used in linear regression, the problem is called **Ordinary Least Squares (OLS)**.

The optimization objective is:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \ \ell(\mathbf{w})$$

Convexity of MSE

Before solving, we establish that this optimization problem has a unique global solution.

Definition 27 (Convex Function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is **convex** if for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $\lambda \in [0, 1]$:

$$f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda)f(\mathbf{y})$$

Geometrically: the line segment between any two points on the graph lies on or above the graph.

Remark 24 (Why Convexity Matters)**.** For convex optimization problems, **every local minimum is a global minimum**. The MSE loss $\ell(\mathbf{w})$ is a quadratic function of $\mathbf{w}$ (a paraboloid opening upward), hence convex. Setting the gradient to zero is therefore sufficient to find the unique optimal $\mathbf{w}^*$.

Closed-Form Solution: The Normal Equations

Since $\ell(\mathbf{w})$ is convex, we find the minimum by setting the gradient to zero.

Step 1: Expand the Loss

$$\ell(\mathbf{w}) = \frac{1}{N}\left(\mathbf{Y}^T\mathbf{Y} - 2\mathbf{w}^T\tilde{\mathbf{X}}^T\mathbf{Y} + \mathbf{w}^T\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}\mathbf{w}\right)$$

Step 2: Compute the Gradient

$$\nabla_{\mathbf{w}}\ell(\mathbf{w}) = \frac{2}{N}\left(-\tilde{\mathbf{X}}^T\mathbf{Y} + \tilde{\mathbf{X}}^T\tilde{\mathbf{X}}\mathbf{w}\right)$$

Step 3: Set Gradient to Zero and Solve

$$\nabla_{\mathbf{w}}\ell(\mathbf{w}) = \mathbf{0} \implies \tilde{\mathbf{X}}^T\tilde{\mathbf{X}}\mathbf{w} = \tilde{\mathbf{X}}^T\mathbf{Y}$$

$$\implies \boxed{\mathbf{w}^* = \left(\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}\right)^{-1}\tilde{\mathbf{X}}^T\mathbf{Y}}$$

The equation $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}\mathbf{w} = \tilde{\mathbf{X}}^T\mathbf{Y}$ is called the **Normal Equation**—a system of $(d+1)$ linear equations in $(d+1)$ unknowns.

OLS in Practice

```python
# OLS regression with sklearn
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()        # uses OLS internally
model.fit(X_train, y_train)       # solve normal equations
y_pred = model.predict(X_test)

print(f"Intercept (w0): {model.intercept_:.4f}")
print(f"Weights:        {model.coef_}")
```

```python
# Manual normal equation: w = (X^T X)^{-1} X^T y
import numpy as np

# Augment X with a column of ones for the bias term
X_aug = np.column_stack([np.ones(len(X_train)), X_train])

# Solve closed-form OLS
w_star = np.linalg.inv(X_aug.T @ X_aug) @ X_aug.T @ y_train

print(f"Bias w0 = {w_star[0]:.4f}")
print(f"Weights = {w_star[1:]}")
```

```python
# Evaluate with R^2 score (coefficient of determination)
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)         # using sklearn
r2_manual = model.score(X_test, y_test)  # equivalent

print(f"R^2 score: {r2:.4f}")
# R^2 = 1: perfect fit, R^2 = 0: predicts the mean
```

Remark 25 (R^2 Interpretation)**.** The **coefficient of determination** $R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$ measures the fraction of variance in y explained by the model. $R^2 = 1$ is a perfect fit; $R^2 = 0$ means the model does no better than predicting the mean of y.

Practical Issues with OLS

The closed-form solution $\mathbf{w}^* = (\tilde{\mathbf{X}}^T\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^T\mathbf{Y}$ is elegant but has three important limitations:

Issue 1: Invertibility of $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}$

The $(d+1) \times (d+1)$ matrix $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}$ must be invertible (full column rank). It fails when:

- **Linearly dependent features:** e.g., including both *net income* and *gross income* (one is a linear function of the other). The columns of $\tilde{\mathbf{X}}$ become linearly dependent, making $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}$ singular.
- **Near-linear dependence (multicollinearity):** The determinant approaches zero, causing numerical instability and unreliable weight estimates.
- **Insufficient data ($N < d+1$):** When there are fewer samples than parameters, the design matrix is "fat" ($\text{rank}(\tilde{\mathbf{X}}) \leq N < d+1$), so $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}}$ cannot have full rank.

Solutions: Use the **Moore–Penrose pseudo-inverse** (`np.linalg.pinv`), remove redundant features, or add **regularization** (Ridge/Lasso), which makes the matrix $\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} + \lambda\mathbf{I}$ always invertible.

Issue 2: Computational Cost

Matrix inversion is $O(d^3)$. For thousands or millions of features, the closed-form becomes prohibitively expensive.

Solution: Use iterative optimization via **Gradient Descent**, which scales as $O(Nd)$ per step and avoids explicit inversion entirely.

Issue 3: Overfitting

Minimizing training error alone does not guarantee generalization. With enough features, OLS can perfectly fit training data (zero training loss) while performing poorly on unseen data.

Solution: Regularization penalizes model complexity (large weights), and techniques like cross-validation estimate the true generalization error.

k-Nearest Neighbors and Classification Metrics

Instance-Based Learning

Every model we have studied so far—linear regression, logistic regression, SVM—learns an explicit set of parameters $\mathbf{w}^*$ during training and then discards the data. **k-Nearest Neighbors (kNN)** takes the opposite approach: it stores *all* training examples and defers computation to prediction time. This paradigm is called **instance-based** (or **lazy**) **learning**.

Definition 28 (k-Nearest Neighbors)**.** Given a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ and a query point $\mathbf{x}$:

1. Compute the distance $d(\mathbf{x}, \mathbf{x}^{(i)})$ to every training example.
2. Identify the set $\mathcal{N}_k(\mathbf{x})$ of the k closest training examples.
3. **Classification:** assign $\mathbf{x}$ the class receiving the most votes (majority rule):

$$\hat{y} = \arg\max_c \sum_{\mathbf{x}^{(i)} \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y^{(i)} = c]$$

4. **Regression:** return the average of the neighbors' target values:

$$\hat{y} = \frac{1}{k} \sum_{\mathbf{x}^{(i)} \in \mathcal{N}_k(\mathbf{x})} y^{(i)}$$

kNN is **non-parametric**: it makes no assumptions about the functional form of the decision boundary or the underlying data distribution. The model *is* the dataset itself.

Distance Metrics

The notion of "closest" depends entirely on the chosen distance metric. For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$:

Definition 29 (Minkowski Distance (L_p Norm))**.**

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p}$$

Euclidean distance ($p = 2$): $d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}$. The most common default; corresponds to the L_2 norm.

Manhattan distance ($p = 1$): $d_1(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^d |x_j - y_j|$. Sums absolute differences along each axis (L_1 norm). More robust to outliers than L_2.

Chebyshev distance ($p \to \infty$): $d_\infty(\mathbf{x}, \mathbf{y}) = \max_j |x_j - y_j|$. Only the largest coordinate difference matters.

Remark 26 (Feature Scaling is Mandatory)**.** If feature x_1 ranges over $[0, 1000]$ and x_2 over $[0, 1]$, then x_1 will dominate every distance computation. You **must** normalize or standardize features before applying kNN—otherwise the metric effectively ignores low-variance features.

The Hyperparameter k

The single hyperparameter k controls the **bias–variance trade-off** of kNN:

$k = 1$: Every query point is assigned the label of its single nearest neighbor. The model captures all local structure in the data—including noise. This yields **high variance** (overfitting). On training data, k=1 always achieves 100% accuracy, which is a clear sign it memorizes rather than generalizes.

Large k: The decision is averaged over many neighbors, smoothing out noise. This introduces **high bias** (underfitting). In the extreme, $k = N$ assigns every point to the global majority class.

Even k: Can produce ties in binary classification. Use **odd** k or implement tie-breaking (e.g., prefer the class of the nearest neighbor among the tied set).

Remark 27 (Selecting k)**.** Treat k as a hyperparameter and select it via **cross-validation**. Plot validation accuracy for $k = 1, 3, 5, \ldots$ and pick the k that maximizes generalization performance. Note that *smaller* k corresponds to *higher* model complexity—the relationship is **inverse**.

Decision Boundaries

For k=1, the decision boundary is determined by the **Voronoi tessellation** of the training points: each training example "owns" the region of feature space closest to it, and the boundaries between differently-labeled regions form piecewise-linear surfaces. This produces highly complex, jagged boundaries that track every noise point.

As k increases, the majority-vote averaging smooths these boundaries. The decision regions become larger, simpler, and more regular. This is a direct visual manifestation of the bias–variance trade-off: small k yields intricate boundaries that fit noise; large k yields blunt boundaries that may miss true local structure.

Strengths and Weaknesses

Strengths:

- Conceptually simple—no training phase, no parameters to optimize.
- Naturally handles **multi-class** problems without modification.
- Can learn arbitrarily complex, **non-linear** decision boundaries.
- Adapts immediately when new data is added (no retraining).

Weaknesses:

- **Slow prediction:** every query requires computing N distances in $\mathbb{R}^d$, giving $O(Nd)$ per prediction.
- **Memory-intensive:** the entire training set must be stored.
- **Sensitive to irrelevant features:** noisy or uninformative dimensions distort distances, degrading performance.
- **Curse of dimensionality:** in high-dimensional spaces, distances between all points converge, making the concept of "nearest" meaningless. kNN degrades rapidly as d grows.

kNN in Practice with `scikit-learn`

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

# Load data and split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Always scale features before kNN!
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# Fit and evaluate
knn = KNeighborsClassifier(n_neighbors=5, weights='uniform')
knn.fit(X_train, y_train)
print(f"Test accuracy: {knn.score(X_test, y_test):.3f}")
```

Selecting k via Cross-Validation

```
from sklearn.model_selection import cross_val_score
import numpy as np

k_values = range(1, 26, 2)          # odd values 1..25
cv_scores = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5)
    cv_scores.append(scores.mean())

best_k = list(k_values)[np.argmax(cv_scores)]
print(f"Best k = {best_k}, CV accuracy = {max(cv_scores):.3f}")
```

Classification Metrics

Accuracy alone can be deeply misleading. Consider a medical screening test where only 1% of patients have the disease: a model that *always* predicts "healthy" achieves 99% accuracy yet catches zero true cases. We need richer metrics that separate different *types* of errors.

The Confusion Matrix

For binary classification with $y \in \{0, 1\}$ (negative/positive), every prediction falls into one of four categories:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

TP: Correctly predicted positive ("hit").

TN: Correctly predicted negative.

FP: Incorrectly predicted positive ("false alarm" / Type I error).

FN: Incorrectly predicted negative ("miss" / Type II error).

Core Metrics

Definition 30 (Accuracy)**.**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The fraction of all predictions that are correct. Misleading when classes are **imbalanced**.

Definition 31 (Precision)**.**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Of all examples *predicted* positive, what fraction is truly positive? High precision means few false alarms.

Definition 32 (Recall (Sensitivity))**.**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Of all *actually* positive examples, what fraction did we detect? High recall means few missed positives.

Definition 33 (F1 Score)**.**

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The harmonic mean of precision and recall. It is high only when **both** precision and recall are high; it penalizes extreme imbalance between the two more harshly than an arithmetic mean would.

Definition 34 (Specificity)**.**

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Of all *actually* negative examples, what fraction did we correctly identify as negative? Specificity is the "recall for the negative class."

The Precision–Recall Trade-Off

Precision and recall are inherently in tension. Increasing the classifier's threshold (requiring higher confidence to predict positive) increases precision but decreases recall, and vice versa. The right balance depends on the application:

High Recall critical: Medical diagnosis, fraud detection—missing a true positive is very costly. We prefer to tolerate more false positives (lower precision) rather than miss actual cases.

High Precision critical: Spam filtering, content recommendation—falsely labeling a legitimate email as spam (false positive) is more annoying than letting a few spam emails through (false negatives).

Remark 28 (Multi-class Extension)**.** For problems with $C > 2$ classes, compute precision, recall, and F1 *per class* (treating each class as "positive" vs. all others). These per-class scores can be combined via **macro-averaging** (unweighted mean across classes) or **weighted averaging** (weighted by class support).

Computing Metrics with `scikit-learn`

```
from sklearn.metrics import (confusion_matrix,
    precision_score, recall_score, f1_score,
    classification_report)

# Assuming y_test and y_pred are already computed
y_pred = knn.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# For multi-class: average='macro' treats all classes equally
print(f"Precision : {precision_score(y_test, y_pred, average='macro'):.3f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='macro'):.3f}")
print(f"F1 Score  : {f1_score(y_test, y_pred, average='macro'):.3f}")

# Full per-class breakdown
print(classification_report(y_test, y_pred,
      target_names=['setosa', 'versicolor', 'virginica']))
```

Outliers: Detection and Handling

What Are Outliers?

Definition 35 (Outlier)**.** An **outlier** (or **anomaly**) is an observation that deviates significantly from the large majority of other observations in a dataset. Formally, given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}\}$, a point $\mathbf{x}^{(i)}$ is an outlier if it lies in a low-probability region of the data-generating distribution $P(\mathbf{x})$.

Outliers are not inherently bad. They fall into two broad categories:

Erroneous outliers: Artifacts of data entry mistakes, sensor malfunctions, software bugs, transcription errors, or sampling from wrong sources. These should typically be removed or corrected.

Genuine outliers: True but rare phenomena—a legitimate signal in the data (e.g., a fraudulent transaction, a once-in-a-century flood, Messi in the population of soccer players). These are often the most *interesting* points and should be preserved.

Remark 29 (Impact on ML Models)**.** Outliers distort sample means, inflate variance estimates, skew regression lines, and degrade model performance. However, not all models are equally vulnerable:

- **Highly sensitive:** Linear regression (OLS), kNN, SVMs with RBF kernels, PCA.
- **More robust:** Tree-based models (decision trees, random forests, gradient boosting), models trained with L_1 loss, median-based estimators.

A single outlier can shift the OLS regression line dramatically because the squared loss $\ell = (y - \hat{y})^2$ amplifies large residuals. Tree-based models split on rank-order statistics, making them largely indifferent to extreme values.

Types of Outliers

Global outlier: A point that deviates significantly from *the entire dataset*. Example: a salary of \$10M in a dataset where most values are \$40K–\$120K.

Contextual outlier: A point that is anomalous only within a specific context. Example: a temperature of 35°C is normal in summer but anomalous in winter for the same city.

Collective outlier: A group of observations that are individually normal but collectively anomalous. Example: a sudden burst of 100 identical purchases within one second.

Detecting Outliers

Detection methods split into three families: **visual**, **statistical**, and **model-based**.

Visual Methods

Quick, exploratory, and useful before any formal analysis:

Box plots: Display the median, quartiles (Q_1, Q_3), and **whiskers** extending to $Q_1 - 1.5 \cdot \text{IQR}$ and $Q_3 + 1.5 \cdot \text{IQR}$, where $\text{IQR} = Q_3 - Q_1$. Points beyond the whiskers are flagged as outliers.

Scatter plots: In two dimensions, outliers appear as isolated points far from the main cluster. Useful for spotting multivariate outliers that per-feature methods miss.

Histograms: Long tails or isolated bars far from the bulk of the distribution signal potential outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({"value": [10, 12, 12, 13, 12, 11, 14, 13, 15, 102]})
df.boxplot(column="value")          # Outlier (102) appears as a dot
plt.title("Box Plot Outlier Detection")
plt.show()
```

Statistical Methods

Z-Score Method (Parametric)

Definition 36 (Z-Score)**.** The **z-score** of an observation x standardizes it relative to the sample mean μ and standard deviation σ:

$$z = \frac{x - \mu}{\sigma}$$

A common rule flags x as an outlier if $|z| > 3$, since under a normal distribution $\approx 99.7\%$ of data falls within $\pm 3\sigma$.

Remark 30 (Limitation)**.** The z-score method **assumes approximate normality**. It is also self-defeating: outliers inflate μ and σ, which can *mask* other outliers (the masking effect). For heavy-tailed or skewed distributions, prefer the modified z-score or IQR method.

```
import numpy as np

data = np.array([10, 12, 12, 13, 12, 11, 14, 13, 15, 102])
z_scores = (data - data.mean()) / data.std()
outliers = data[np.abs(z_scores) > 3]   # Points with |z| > 3
print(f"Outliers: {outliers}")           # [102]
```

Modified Z-Score (Robust)

Replace mean and standard deviation with their robust counterparts—the **median** and the **Median Absolute Deviation (MAD)**:

$$\text{MAD} = \text{median}\left(|x_i - \widetilde{x}|\right), \qquad M_i = \frac{0.6745\,(x_i - \widetilde{x})}{\text{MAD}}$$

where $\widetilde{x}$ is the median and 0.6745 is the z-value at the 75th percentile of the standard normal. Flag if $|M_i| > 3.5$. Because the median is unaffected by extreme values, this method resists the masking effect.

IQR Method (Non-Parametric)

Definition 37 (Interquartile Range)**.** The **IQR** is the difference between the third and first quartiles:

$$\text{IQR} = Q_3 - Q_1$$

The "normal" range is defined as $[Q_1 - k \cdot \text{IQR},\ Q_3 + k \cdot \text{IQR}]$ with $k = 1.5$ for standard outliers and $k = 3$ for extreme outliers. Any observation outside this range is flagged.

This method makes **no distributional assumptions** and is robust to skewness. It is the method underlying box plot whiskers.

```
import pandas as pd

s = pd.Series([10, 12, 12, 13, 12, 11, 14, 13, 15, 102])
Q1, Q3 = s.quantile(0.25), s.quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
clean = s[(s >= lower) & (s <= upper)]  # Remove outliers
print(f"Bounds: [{lower:.1f}, {upper:.1f}]")
print(f"Kept {len(clean)}/{len(s)} points")
```

Percentile-Based Clipping

A simpler non-parametric approach: flag values below the 1st percentile or above the 99th percentile. This is distribution-free but discards a fixed fraction of data regardless of whether true outliers exist.

Model-Based Methods

When statistical methods are insufficient—especially in high-dimensional or non-Gaussian settings—ML models can learn the boundary between normal and anomalous.

Isolation Forest: Builds an ensemble of random trees that recursively partition feature space by random splits. Outliers are *easier to isolate*: they require fewer splits to be separated from the rest. The **anomaly score** is proportional to the average path length from root to the isolated point—shorter paths indicate higher anomaly.

Local Outlier Factor (LOF): Compares the **local density** of a point to the density of its k-nearest neighbors. A point whose neighborhood is much sparser than its neighbors' neighborhoods receives a high LOF score ($\gg 1$). This detects outliers that are anomalous *relative to their local context*, unlike global methods.

DBSCAN: A density-based clustering algorithm. Points in dense regions are assigned to clusters; points that do not belong to any cluster (i.e., fall in low-density regions) are labeled as noise/outliers. No predefined number of clusters is needed.

```
from sklearn.ensemble import IsolationForest
import numpy as np

np.random.seed(42)
X = np.random.randn(100, 2)             # Normal data
X = np.vstack([X, [[10, 10], [-8, -8]]])  # Add 2 outliers

iso = IsolationForest(contamination=0.02, random_state=42)
labels = iso.fit_predict(X)              # 1 = inlier, -1 = outlier
outliers = X[labels == -1]
print(f"Detected {len(outliers)} outliers")
```

Handling Outliers

Detection alone is not enough—you must decide what to *do* with flagged points. The right action depends on **why** the outlier exists.

Remove: Delete the outlier rows entirely. Appropriate when outliers are clearly erroneous (e.g., a negative age, a temperature of 9999). Risky if the "outlier" is a genuine rare event.

Cap / Winsorize: Clip extreme values to a chosen percentile boundary (e.g., cap at the 1st and 99th percentiles). Preserves the observation while limiting its influence. Named after biostatistician Charles Winsor.

Transform: Apply a concave transformation—$\log(x)$, $\sqrt{x}$, or Box-Cox—to compress the right tail and reduce skewness. This pulls extreme values closer to the bulk without discarding them.

Impute: Replace the outlier value with a robust statistic such as the median or the nearest IQR boundary. Useful when you cannot afford to lose the row (e.g., multivariate data where other features are valid).

Use robust models: Instead of fixing the data, choose models that are inherently less sensitive: tree-based models, Huber loss regression, L_1 (least absolute deviations) regression, or median regression (quantile regression at $\tau = 0.5$).

Keep: If the outlier is genuine and informative—a fraud case, a rare disease, a system anomaly—**keep it**. Removing real signal degrades model quality. Consider gathering more data around the rare event or using anomaly-aware models.

Remark 31 (Decision Heuristic)**.** Ask three questions: (1) Is the outlier a data error? $\rightarrow$ Remove or impute. (2) Is it a real but irrelevant extreme? $\rightarrow$ Cap or transform. (3) Is it a real and informative event? $\rightarrow$ Keep it, and ensure your model can handle it.

Feature-Wise vs. Multivariate Outliers

All methods discussed so far can be applied per-feature (**univariate analysis**). However, a critical subtlety arises in multidimensional data:

Remark 32 (Multivariate Outliers)**.** A point can appear perfectly normal in every individual feature yet be an outlier *in combination*. For example, a person who is 190 cm tall and weighs 55 kg: both height and weight are individually within normal ranges, but their combination is unusual.

Definition 38 (Mahalanobis Distance)**.** The **Mahalanobis distance** measures how far a point $\mathbf{x}$ is from the center of a distribution, accounting for correlations between features:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

where $\boldsymbol{\mu}$ is the mean vector and $\boldsymbol{\Sigma}$ is the covariance matrix. Under multivariate normality, D_M^2 follows a χ^2 distribution with d degrees of freedom, providing a principled threshold for flagging multivariate outliers.

Univariate methods applied feature-by-feature will **miss** these multivariate outliers. When features are correlated, always consider multivariate detection methods: Mahalanobis distance, Isolation Forest (which naturally operates in full feature space), or LOF.

Summary: Detection Method Selection

Method	**Assumptions**	**Best For**
Z-Score	Normal distribution	Low-dimensional, Gaussian data
Modified Z-Score	None (uses median/MAD)	Skewed or contaminated data
IQR / Box plot	None	Quick exploratory analysis
Percentile clipping	None	Simple, fixed-fraction trimming
Isolation Forest	None	High-dimensional, mixed types
LOF	None	Local density variations
DBSCAN	Density-reachable clusters	Cluster-based anomaly detection
Mahalanobis	Multivariate normal	Correlated features

Feature Scaling

Why Feature Scaling Matters

Real-world datasets contain features measured in different units and spanning wildly different ranges. Consider a dataset with *age* (0–100) and *salary* ($20,000–$200,000): a raw Euclidean distance computation is dominated almost entirely by salary, making age nearly invisible to the model.

Remark 33 (Which Algorithms Are Affected?)**.** Not every algorithm suffers from unscaled features. The key distinction is whether the algorithm relies on **distances** or **gradient-based optimization**:

- **Distance-based methods** (k-NN, SVM, k-Means): the feature with the largest range dominates the distance metric, biasing the model toward that single feature.
- **Gradient descent**: unscaled features produce elongated loss contours, causing the optimizer to zig-zag slowly toward the minimum instead of descending directly.
- **Regularization** (L_1, L_2): penalizes all weights equally. If features live on different scales, the penalty is not comparable across weights — a large weight on a tiny-range feature is not the same as a large weight on a huge-range feature.
- **Tree-based methods** (Decision Trees, Random Forests, Gradient Boosting): **not affected**, because splits depend only on feature ordering, not magnitude.

Standardization (Z-Score Normalization)

Definition 39 (Standardization)**.** Given a feature with sample mean μ and sample standard deviation σ, **standardization** transforms each value x into a **z-score**:

$$x' = \frac{x - \mu}{\sigma}$$

The resulting feature has **zero mean** and **unit variance**.

Properties:

- Linear transformation — does not change the shape of the distribution.
- Output is *not* bounded to any fixed range.
- Works well for approximately **normally distributed** features, but applies to any distribution.

Preferred for: SVM, Logistic Regression, PCA, Neural Networks.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit + transform
X_test_scaled  = scaler.transform(X_test)        # Transform only!

# Verify: training set has mean~0, std~1
print(X_train_scaled.mean(axis=0))  # array of ~0s
print(X_train_scaled.std(axis=0))   # array of ~1s
```

Min-Max Normalization

Definition 40 (Min-Max Scaling)**.** **Min-Max normalization** rescales each feature to a fixed range $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the feature.

Properties:

- Preserves the original distribution shape (linear rescaling to $[0, 1]$).
- Sensitive to **outliers**: a single extreme value compresses all other values into a narrow sub-range.

Preferred for: Neural network input layers, image pixel scaling ($[0, 255] \to [0, 1]$), algorithms that expect bounded inputs.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()                          # Default range [0,1]
X_train_scaled = scaler.fit_transform(X_train)   # Fit on train
X_test_scaled  = scaler.transform(X_test)        # Same params on test

# Custom range example:
scaler_custom = MinMaxScaler(feature_range=(-1, 1))
X_train_custom = scaler_custom.fit_transform(X_train)
```

Robust Scaling

Definition 41 (Robust Scaling)**. Robust scaling** centers by the **median** and scales by the **interquartile range** ($\text{IQR} = Q_3 - Q_1$):

$$x' = \frac{x - \text{median}}{Q_3 - Q_1}$$

Because the median and IQR are resistant to extreme values, robust scaling is the preferred choice when the data contains **significant outliers** that would distort standardization or min-max scaling.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

MaxAbs Scaling

Definition 42 (MaxAbs Scaling)**. MaxAbs scaling** divides each feature by its maximum absolute value:

$$x' = \frac{x}{|x_{\max}|}$$

The result is bounded to $[-1, 1]$.

The critical property of MaxAbs scaling is that it **preserves sparsity**: zero entries remain exactly zero after transformation. This makes it the natural choice for **sparse data** (e.g., TF-IDF text features, one-hot encodings).

Comparison of Scaling Methods

Method	Formula	Range	Outlier-Robust?
Standard	$(x - \mu)/\sigma$	Unbounded	No
Min-Max	$(x - x_{\min})/(x_{\max} - x_{\min})$	$[0, 1]$	No
Robust	$(x - \text{median})/\text{IQR}$	Unbounded	Yes
MaxAbs	$x/\lvert x_{\max}\rvert$	$[-1, 1]$	No

Critical Rule: Fit on Training Data Only

Remark 34 (Information Leakage)**.** The scaler's parameters (μ, σ, $x_{\min}$, $x_{\max}$, etc.) must be computed **exclusively from the training set**. The same learned parameters are then applied to the validation and test sets via `.transform()`.

Never call `fit()` or `fit_transform()` on the test set. Doing so leaks information about the test distribution into the preprocessing step, producing optimistically biased evaluation metrics.

The correct workflow:

1. `scaler.fit_transform(X_train)` — learn parameters from training data and transform it.
2. `scaler.transform(X_val)` — apply the *same* parameters to validation data.
3. `scaler.transform(X_test)` — apply the *same* parameters to test data.

Using a Pipeline to Prevent Leakage

An `sklearn.pipeline.Pipeline` bundles the scaler and the model into a single estimator, ensuring that `fit_transform` is called on training folds and `transform` on validation folds automatically during cross-validation.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

# Bundle scaler + model: scaling is handled correctly
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('knn',    KNeighborsClassifier(n_neighbors=5))
])

# Cross-validation applies fit_transform on train folds,
# transform on validation folds -- no leakage
scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print(f"Mean CV Accuracy: {scores.mean():.3f}")

# Final fit on all training data, evaluate on held-out test set
pipe.fit(X_train, y_train)
print(f"Test Accuracy: {pipe.score(X_test, y_test):.3f}")
```

When to Scale

Always scale: k-NN, SVM, Logistic Regression, Neural Networks, PCA, k-Means, and any method using gradient descent or distance computations.

No need to scale: Decision Trees, Random Forests, Gradient Boosted Trees — splits depend on feature order, not magnitude.

Order of operations: Apply scaling **after** handling missing values and outliers, since the scaler's statistics (μ, σ, etc.) would be corrupted by placeholder values or extreme outliers.

Correlations, Polynomial Features, and Types of Feature Data

Before a model sees a single training example, the quality of its features has already placed a ceiling on its performance. This chapter addresses three pillars of feature engineering: understanding *what kind* of data each feature carries, detecting *redundancy* via correlations, and expanding *expressive power* through polynomial features.

Types of Feature Data

Every column in a dataset belongs to one of two broad families: **numerical** or **categorical**. The distinction matters because most ML algorithms operate on real-valued vectors—categorical data must be converted before it can enter the pipeline.

Numerical Features

Continuous: Can take any real value within a range (possibly infinite). Examples: temperature, price, height, blood pressure. Measured on a ratio or interval scale.

Discrete: Take on countable values, typically non-negative integers. Examples: number of rooms, word count, customer visits. Often arise from counting processes.

Numerical features can be fed directly into linear models, distance-based methods, and neural networks (after appropriate scaling).

Categorical Features

Nominal: Categories with *no intrinsic order*. Examples: color $\in$ {red, blue, green}, country, blood type. Assigning integers (red= 0, blue= 1, green= 2) is dangerous because it fabricates an ordering the data does not possess.

Ordinal: Categories with a *natural ordering*, but non-uniform spacing. Examples: education level (HS $<$ BS $<$ MS $<$ PhD), satisfaction rating (low $<$ medium $<$ high). Integer encoding is permissible here because the ordering is meaningful.

Remark 35 (Why the Distinction Matters)**.** If a nominal feature like `color` is naively encoded as $\{0, 1, 2\}$, a linear model will interpret $\text{green} - \text{red} = 2 \cdot (\text{blue} - \text{red})$—a relationship that is pure artifact. Ordinal features tolerate integer encoding precisely because their ordering is real.

Encoding Categorical Features

Label Encoding (Integer Mapping)

Map each category to a distinct integer: $\text{HS} \to 0,\ \text{BS} \to 1,\ \text{MS} \to 2,\ \text{PhD} \to 3$.

- **Appropriate for ordinal data only**, where the integer ordering reflects reality.
- For nominal data, this introduces a spurious metric structure.

```
import pandas as pd

df = pd.DataFrame({'education': ['HS', 'BS', 'MS', 'PhD', 'BS']})
# Ordinal encoding: map preserves the natural order
order_map = {'HS': 0, 'BS': 1, 'MS': 2, 'PhD': 3}
df['edu_encoded'] = df['education'].map(order_map)
print(df)
```

One-Hot Encoding (Dummy Variables)

For **nominal** features, create K binary indicator columns (one per category), where exactly one column is 1 for each sample.

For a feature `color` $\in$ {red, blue, green}:

$$\text{red} \to [1,0,0], \quad \text{blue} \to [0,1,0], \quad \text{green} \to [0,0,1]$$

Remark 36 (Dummy Variable Trap)**.** If K one-hot columns sum to 1 for every sample, the columns are linearly dependent. When using models with an intercept (e.g., OLS), this causes **perfect multicollinearity**. The fix: drop one column, using $K-1$ indicators. The dropped category becomes the implicit *reference level*.

```
import pandas as pd

df = pd.DataFrame({'color': ['red', 'blue', 'green', 'blue']})
# One-hot encode, drop_first avoids the dummy variable trap
dummies = pd.get_dummies(df['color'], drop_first=True, dtype=int)
print(dummies)
#    green  red
# 0      0    1
# 1      0    0    <- "blue" is the dropped reference
# 2      1    0
# 3      0    0
```

Equivalently, using scikit-learn (useful inside pipelines):

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

enc = OneHotEncoder(drop='first', sparse_output=False)
X_cat = np.array([['red'], ['blue'], ['green'], ['blue']])
X_enc = enc.fit_transform(X_cat)   # shape (4, 2)
print(enc.categories_)             # [array(['blue','green','red'])]
```

Target (Mean) Encoding

Replace each category with the *mean of the target variable* for that category. For a feature `city` predicting `price`: $\text{city}_k \to \bar{y}_k = \frac{1}{|D_k|}\sum_{i \in D_k} y^{(i)}$.

- Produces a single numeric column regardless of cardinality—useful for high-cardinality features.
- **Risk of data leakage:** the encoding uses target information. Must be computed on training folds only (never on test data) or use smoothing/regularization to avoid overfitting to rare categories.

Correlation Analysis

Correlations serve two purposes in feature engineering: (1) identifying **redundant features** (highly correlated with each other), and (2) identifying **predictive features** (highly correlated with the target).

Pearson Correlation Coefficient

Definition 43 (Pearson Correlation Coefficient)**.** For two random variables X and Y with standard deviations $\sigma_X, \sigma_Y > 0$, the Pearson correlation coefficient is:

$$r_{XY} = \frac{\text{Cov}(X,Y)}{\sigma_X \cdot \sigma_Y} = \frac{\displaystyle\sum_{i=1}^{m}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\displaystyle\sum_{i=1}^{m}(x_i - \bar{x})^2} \cdot \sqrt{\displaystyle\sum_{i=1}^{m}(y_i - \bar{y})^2}}$$

where $r_{XY} \in [-1, +1]$.

$r = +1$**:** Perfect positive linear relationship—Y increases proportionally with X.

$r = -1$**:** Perfect negative linear relationship—Y decreases proportionally as X increases.

$r \approx 0$**:** No *linear* relationship. But X and Y may still be strongly related in a non-linear way (e.g., $Y = X^2$ gives $r \approx 0$ for symmetric X).

The absolute value $|r|$ measures relationship **strength**; the sign indicates **direction**.

Correlation Matrix and Feature Selection

The **correlation matrix** $C \in \mathbb{R}^{d \times d}$ stores pairwise Pearson coefficients for all d features. It is symmetric with $C_{ii} = 1$ along the diagonal. Visualized as a heatmap, it immediately reveals clusters of correlated features.

```
import pandas as pd
import seaborn as sb
import matplotlib.pyplot as plt

df = pd.read_csv('housing.csv')    # example dataset
corr = df.corr()                   # pairwise Pearson correlations

plt.figure(figsize=(10, 8))
sb.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Feature Correlation Matrix')
plt.tight_layout(); plt.show()
```

Practical usage of the correlation matrix:

- **Multicollinearity detection:** If $|r_{ij}| > 0.8$ for features i and j, they are nearly redundant. Keeping both inflates model complexity, slows convergence, and causes numerical instability in OLS (recall the matrix inversion in $\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$). Drop one.
- **Feature–target correlation:** Features with $|r|$ close to 0 against the target contribute little predictive signal to a linear model and are candidates for removal.
- **Example:** In the Boston Housing dataset, `RAD` and `TAX` have $r = 0.91$—effectively encoding the same information. Selecting `RM` ($r = 0.70$ with target) and `LSTAT` ($r = -0.74$ with target) as predictors captures the most information with minimal redundancy.

Remark 37 (Correlation $\neq$ Causation)**.** A high correlation between X and Y does not imply that X causes Y. Both may be driven by a latent confounding variable, or the correlation may be coincidental. Correlation is a *statistical* observation; causation requires controlled experimentation or causal inference frameworks.

Remark 38 (Beyond Pearson)**.** Pearson's r captures only **linear** associations. For monotonic but non-linear relationships, **Spearman's rank correlation** (operates on ranks rather than raw values) is more robust. Pandas offers both via `df.corr(method='spearman')`.

Polynomial Features

When the true relationship between features and target is non-linear, a standard linear model $\hat{y} = \mathbf{w}^T \mathbf{x}$ will underfit. **Polynomial features** address this by expanding the input space so that a *linear* model in the expanded space corresponds to a *non-linear* model in the original space.

Single-Feature Case

Given a single feature x, we construct the polynomial feature vector of degree M:

$$\phi(x) = [1,\ x,\ x^2,\ x^3,\ \ldots,\ x^M]^T$$

The polynomial regression model is:

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \mathbf{w}^T \phi(x)$$

Remark 39 (Still Linear in Parameters)**.** Despite the non-linear feature transformations, the model is **linear in the weights $\mathbf{w}$**. This means we can use the exact same OLS machinery, gradient descent, and regularization techniques developed for linear regression. The "trick" is that we transform the input *before* feeding it to a linear learner.

Multi-Feature Case and Interaction Terms

For d original features $[x_1, x_2, \ldots, x_d]$ and degree $M = 2$, `PolynomialFeatures` generates:

$$\phi(\mathbf{x}) = [1,\ x_1,\ x_2,\ \ldots,\ x_d,\ x_1^2,\ x_1x_2,\ \ldots,\ x_d^2]$$

The **interaction terms** (e.g., x_1x_2) capture joint effects that individual features cannot represent. The number of output features for degree M with d inputs is $\binom{d+M}{M}$, which grows combinatorially—a practical concern for large d.

Bias–Variance Trade-off

Low degree (M small): High bias (underfitting). The model cannot capture complex relationships.

High degree (M large): High variance (overfitting). The model fits noise in the training data, and the large number of parameters relative to samples leads to poor generalization.

Mitigation strategies: use **regularization** (Ridge, Lasso) to penalize large coefficients, and select M via **cross-validation**.

Polynomial Regression in Practice

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

# Generate non-linear data: y = 3x - 0.5x^2 + noise
np.random.seed(42)
X = np.sort(np.random.uniform(-3, 3, 50)).reshape(-1, 1)
y = 3*X.ravel() - 0.5*X.ravel()**2 + np.random.randn(50)*0.5

# Degree-2 polynomial pipeline
poly_model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LinearRegression()
)
poly_model.fit(X, y)
y_pred = poly_model.predict(X)

print(f"MSE = {mean_squared_error(y, y_pred):.3f}")
# Inspect learned coefficients: [w1, w2] for [x, x^2]
lr = poly_model.named_steps['linearregression']
print(f"Coefficients: {lr.coef_}")     # ~ [3.0, -0.5]
print(f"Intercept:    {lr.intercept_:.3f}")
```

```
# Comparing degrees 1, 2, 5 on the same data
from sklearn.model_selection import cross_val_score

for deg in [1, 2, 5]:
    pipe = make_pipeline(
        PolynomialFeatures(degree=deg, include_bias=False),
        LinearRegression()
    )
    scores = cross_val_score(pipe, X, y, cv=5,
                             scoring='neg_mean_squared_error')
    print(f"Degree {deg}: CV-MSE = {-scores.mean():.3f}")
# Degree 2 should win: matches the true data-generating process
```

Remark 40 (Practical Checklist)**.** When applying polynomial features:

1. **Scale first.** Features on different scales produce enormous polynomial values. Standardize before applying `PolynomialFeatures`.
2. **Start low.** Try degree 2 or 3. Degrees above 5 rarely help and almost always overfit.
3. **Regularize.** Combine with Ridge or Lasso to keep weights under control.
4. **Cross-validate.** Let the data select the degree, not intuition.

Tying It All Together

A typical feature engineering pipeline for tabular data:

1. **Identify feature types:** numerical (continuous/discrete) vs. categorical (nominal/ordinal).
2. **Encode categoricals:** label encoding for ordinal, one-hot for nominal (drop one column).
3. **Analyze correlations:** compute the correlation matrix, drop one feature from every pair with $|r| > 0.8$, and prioritize features strongly correlated with the target.
4. **Expand features if needed:** apply polynomial features to capture non-linearities, using cross-validation to choose the degree and regularization to prevent overfitting.

Feature engineering is often where the real predictive power is won or lost. As Andrew Ng put it: *"Applied Machine Learning is basically feature engineering."*

Feature Engineering for Images (Part 1)

Image Representation

Before feeding images into traditional ML models (SVMs, Random Forests, Logistic Regression), we must transform raw pixels into structured, meaningful features. This chapter covers the classical feature engineering pipeline for image data.

Definition 44 (Image as a Tensor)**.** A digital image $\mathbf{I}$ is a discrete, multidimensional grid of numerical values:

$$\mathbf{I} \in \mathbb{R}^{H \times W \times C}$$

where H = height (rows), W = width (columns), and C = number of color channels.

Grayscale: $C = 1$. Each pixel stores a single intensity value, typically $\in [0, 255]$ (8-bit) where 0 is black and 255 is white.

Color (RGB): $C = 3$. Each pixel is a vector (R, G, B) of red, green, and blue intensities. With 8 bits per channel, this yields $256^3 = 16{,}777{,}216$ possible colors.

```
import cv2
import numpy as np
from PIL import Image

# Load image with OpenCV (returns BGR by default)
img_bgr = cv2.imread("photo.jpg")
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
print(img_rgb.shape)   # (H, W, 3) e.g. (480, 640, 3)
print(img_rgb.dtype)   # uint8: values in [0, 255]

# Equivalently with PIL
img_pil = Image.open("photo.jpg")
img_arr = np.array(img_pil)  # Already RGB, shape (H, W, 3)
```

Remark 41. Raw pixel values are highly sensitive to lighting, translation, rotation, and scale. Flattening an image into $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$ and feeding it directly to a model yields poor generalization. We must extract **invariant** and **robust** features.

Image Metadata (EXIF)

Many image formats embed **metadata** via the **Exchangeable Image File Format (EXIF)** standard, recorded by the capture device. These provide features *without* processing pixel data:

Geometric: Height (H), width (W), aspect ratio, resolution.

Photometric: Color space, bit depth (8-bit vs. 16-bit), compression format.

Acquisition: Camera model, shutter speed, aperture, ISO, focal length, GPS coordinates, timestamp.

```
from exif import Image as ExifImage

with open("photo.jpg", "rb") as f:
    meta = ExifImage(f)
if meta.has_exif:
    print(f"Camera: {meta.make} {meta.model}")
    print(f"ISO: {meta.photographic_sensitivity}")
    print(f"GPS: {meta.gps_latitude} {meta.gps_latitude_ref}")
```

Color Spaces and Transformations

RGB channels are heavily correlated with illumination. Transforming the color space is often the first preprocessing step.

Grayscale Conversion: Reduces $C = 3$ to $C = 1$, removing color variance when color is task-irrelevant. The standard luminosity formula (ITU-R BT.601) weights channels by human perceptual sensitivity:

$$Y = 0.299R + 0.587G + 0.114B$$

HSV (Hue, Saturation, Value): Decouples color from brightness.

Hue: Pure color type as an angle $\in [0°, 360°]$.

Saturation: Color purity/intensity.

Value: Brightness level.

Isolating the *Value* channel provides invariance to shadows and lighting while retaining *Hue* for color-based tasks.

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg")

# Convert BGR -> Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape)  # (H, W) -- single channel

# Convert BGR -> HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
hue, sat, val = hsv[:,:,0], hsv[:,:,1], hsv[:,:,2]

# Grayscale intensity range
print(f"Intensity range: [{gray.min()}, {gray.max()}]")
```

Aggregate Statistics (Global Features)

Treat pixel intensities as a distribution and compute its **moments**—a compact global summary of brightness and contrast.

Mean (μ): Average brightness.

$$\mu = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} I(i,j)$$

Variance (σ^2): Intensity spread; correlates with global contrast.

$$\sigma^2 = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} (I(i,j) - \mu)^2$$

Skewness: Asymmetry of the intensity distribution. Positive skew $\Rightarrow$ tail toward brighter intensities.

Kurtosis: "Peakedness" of the distribution. High kurtosis $\Rightarrow$ many pixels far from the mean (high contrast/texture regions).

Remark 42. Aggregate statistics are computationally cheap and invariant to rotation/translation, but discard *all* spatial information. They are typically used as auxiliary features alongside spatial descriptors (SIFT, HOG).

Color Histograms (Global Features)

A **color histogram** captures the global distribution of pixel intensities, completely discarding spatial layout.

Mechanism: Divide the intensity range into discrete bins. Count how many pixels fall into each bin.

Normalization: Divide by total pixel count to obtain a discrete probability distribution, making the feature invariant to image size.

Strengths: Invariant to translation and rotation. Robust to scaling and partial occlusion.

Weakness: Complete loss of spatial information—a red apple on green grass produces the same histogram as randomly scattered red and green pixels.

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Compute grayscale histogram: 256 bins over [0, 255]
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
hist_norm = hist / hist.sum()  # Normalize to probability

# Per-channel color histogram
for i, color in enumerate(["Blue", "Green", "Red"]):
    h = cv2.calcHist([img], [i], None, [256], [0, 256])
    print(f"{color} channel: mean bin count = {h.mean():.1f}")
```

Spatial Patching (Local Features)

To retain spatial context without the rigidity of full-image flattening, divide the image into a grid of **sub-regions (patches)** and compute features (histograms, mean intensity, texture statistics) per patch independently. This produces a feature vector that captures *where* certain visual properties occur, bridging the gap between purely global and purely local representations.

Spatial Filtering and Convolution

To extract structural features—edges, corners, textures—we apply local neighborhood operations by sliding a small weight matrix across the image.

Definition 45 (Kernel / Filter)**.** A **kernel K** is a small matrix of weights (typically 3×3 or 5×5) that defines how a pixel's neighborhood influences its output value.

Cross-Correlation vs. Convolution

Let $\mathbf{I}$ be the image and $\mathbf{K}$ a kernel of size $(2k+1) \times (2k+1)$.

Cross-Correlation ($\otimes$): The kernel slides directly over the image without flipping:

$$G[i,j] = (\mathbf{I} \otimes \mathbf{K})[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} \mathbf{K}[u,v] \cdot \mathbf{I}[i+u, j+v]$$

Convolution ($*$): The kernel is flipped both vertically and horizontally before multiplication. This ensures associativity in signal processing. In ML, the distinction is often irrelevant since kernels are learned:

$$G[i,j] = (\mathbf{I} * \mathbf{K})[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} \mathbf{K}[u,v] \cdot \mathbf{I}[i-u, j-v]$$

Padding and Stride

Padding: Filtering shrinks spatial dimensions. To preserve them, add artificial border pixels: *zero-padding* (zeros), *replicate* (copy edge values), or *reflect* (mirror).

Stride: The step size of the sliding kernel. Stride > 1 produces spatial downsampling.

Common Hand-Crafted Kernels

Filter	3×3 Kernel	Effect
Box Blur	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Uniform neighborhood average. Low-pass filter: removes high-frequency noise but blurs edges.
Gaussian Blur	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	Center-weighted smoothing. More natural, rotationally symmetric blur than box filter.
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Amplifies pixel-neighbor differences. High-pass filter: emphasizes edges and fine details.

```python
import cv2
import numpy as np

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)

# Define and apply a sharpening kernel
sharpen_k = np.array([[ 0, -1,  0],
                      [-1,  5, -1],
                      [ 0, -1,  0]], dtype=np.float32)
sharpened = cv2.filter2D(img, -1, sharpen_k)

# Gaussian blur (5x5 kernel, sigma auto-computed)
blurred = cv2.GaussianBlur(img, (5, 5), 0)
```

Edge Detection and Gradients

Edges are regions of rapid intensity change—they define object boundaries. We detect them by computing the **discrete derivative** of the image.

Image Gradients

The gradient of $\mathbf{I}(x, y)$ is a vector of first-order partial derivatives:

$$\nabla \mathbf{I} = \begin{bmatrix} \frac{\partial \mathbf{I}}{\partial x} \\ \frac{\partial \mathbf{I}}{\partial y} \end{bmatrix}$$

Since images are discrete, derivatives are approximated via **finite differences**, implementable as convolution kernels.

Sobel Operators

The Sobel filters compute the gradient while applying slight Gaussian smoothing to reduce noise:

$$\mathbf{K}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{(detects vertical edges—horizontal gradient)}$$

$$\mathbf{K}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad \text{(detects horizontal edges—vertical gradient)}$$

After computing $G_x = \mathbf{I} * \mathbf{K}_x$ and $G_y = \mathbf{I} * \mathbf{K}_y$, we derive two per-pixel features:

Gradient Magnitude: Edge strength.

$$M = \sqrt{G_x^2 + G_y^2}$$

Gradient Direction: Edge orientation (perpendicular to the edge).

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)

# Sobel gradients (64-bit float to capture negative values)
G_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)  # dI/dx
G_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)  # dI/dy

# Gradient magnitude and direction
magnitude = np.sqrt(G_x**2 + G_y**2)
direction = np.arctan2(G_y, G_x)  # Radians in [-pi, pi]
```

Remark 43. Gradient magnitudes and orientations are the foundation for advanced classical descriptors: **HOG** (Histogram of Oriented Gradients) and **SIFT** (Scale-Invariant Feature Transform), which represent the state-of-the-art in pre-deep-learning computer vision.

Canny Edge Detection Pipeline

Canny is a multi-stage algorithm for producing clean, thin, well-localized edges:

1. **Noise Reduction:** Apply Gaussian blur to suppress noise.
2. **Gradient Calculation:** Compute M and θ via Sobel operators.
3. **Non-Maximum Suppression:** Thin edges by keeping only local maxima along the gradient direction θ—suppresses all non-peak pixels.
4. **Hysteresis Thresholding:** Apply two thresholds T_{high} and T_{low}. Pixels with $M > T_{\text{high}}$ are *strong edges* (always kept). Pixels with $T_{\text{low}} < M < T_{\text{high}}$ are *weak edges*, kept only if connected to a strong edge.

```
import cv2

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)

# Canny edge detection with two thresholds
edges = cv2.Canny(img, threshold1=100, threshold2=200)
# edges is a binary image: 255 = edge, 0 = non-edge

print(f"Edge pixels: {(edges > 0).sum()}")
print(f"Edge density: {(edges > 0).mean():.3f}")
```

Feature Engineering for Images (Part 2)

Image Binarization and Thresholding

Many vision tasks require separating foreground from background before extracting higher-level structure. **Binarization** converts a grayscale image into a binary image by assigning each pixel to one of two classes.

Global Thresholding

A single threshold T is applied uniformly across the entire image:

$$\text{Pixel}_{\text{out}} = \begin{cases} 255 & \text{if Pixel}_{\text{in}} > T \\ 0 & \text{otherwise} \end{cases}$$

```
import cv2
import numpy as np

img = cv2.imread('page.png', cv2.IMREAD_GRAYSCALE)

# Global threshold at T=127: pixels > 127 become 255, else 0
ret, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
```

Remark 44. Global thresholding fails when the image has a lighting gradient (e.g., uneven shadows across a scanned document)—one region turns entirely white while the other turns entirely black.

Adaptive (Local) Thresholding

Instead of a single global T, a **local threshold** is computed independently for each pixel based on a neighborhood window (e.g., an 11×11 block):

Mean Adaptive: Threshold = mean intensity of the local neighborhood.

Gaussian Adaptive: Threshold = Gaussian-weighted sum of neighborhood intensities, giving more influence to nearby pixels.

This naturally handles spatially varying illumination.

```
# Adaptive mean thresholding (blockSize=11, C=2 subtracted from mean)
adapt_mean = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, blockSize=11, C=2
)

# Adaptive Gaussian thresholding
adapt_gauss = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, blockSize=11, C=2
)
```

Otsu's Binarization

Otsu's method automatically discovers the optimal global threshold T^* without manual tuning. It assumes the pixel histogram is **bimodal** (two peaks corresponding to foreground and background) and finds the T that minimizes *intra-class variance*—equivalently, maximizes *inter-class variance*:

$$T^* = \arg\min_T \left[\omega_0(T)\, \sigma_0^2(T) + \omega_1(T)\, \sigma_1^2(T) \right]$$

where $\omega_k(T)$ and $\sigma_k^2(T)$ are the weight (pixel fraction) and variance of class k at threshold T.

```
# Otsu's method: pass 0 as threshold, OpenCV finds optimal T*
ret, otsu = cv2.threshold(
    img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print(f"Otsu's optimal threshold: {ret}")  # e.g., 142.0
```

Corner Detection

To track objects across frames or match features between images, we need to identify points that are **distinct** and **repeatable**. Not all image regions are equally useful:

Flat Regions: No intensity gradient in any direction. Useless for tracking—shifting a window produces no measurable change.

Edges: Strong gradient in one direction only. Subject to the **aperture problem**: we cannot determine position along the edge.

Corners: High gradients in *multiple* directions. Highly distinct and ideal as keypoints.

Harris Corner Detector

The Harris detector identifies corners by analyzing how the **sum of squared differences (SSD)** changes when a local window is shifted by (u, v):

$$E(u,v) = \sum_{(x,y)\in W} \big[I(x+u, y+v) - I(x,y)\big]^2$$

A first-order Taylor expansion yields the approximation $E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$, where M is the **Structure Tensor** (Second Moment Matrix) computed from image derivatives I_x, I_y over window W:

$$M = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

The eigenvalues λ_1, λ_2 of M encode the local gradient structure:

Corner (λ_1 large, λ_2 large): Intensity changes significantly in all shift directions.

Edge ($\lambda_1 \gg \lambda_2$ or vice versa): Change in one direction only.

Flat ($\lambda_1 \approx \lambda_2 \approx 0$): No significant change.

Remark 45. Computing eigenvalues directly is expensive. Harris instead uses the **corner response function**:

$$R = \det(M) - k\,(\mathrm{trace}(M))^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$$

where $k \in [0.04, 0.06]$. A large positive R indicates a corner; $R < 0$ indicates an edge; $|R|$ small indicates a flat region.

```
import cv2
import numpy as np

img = cv2.imread('chessboard.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

# Harris corner detection (blockSize=2, ksize=3, k=0.04)
harris = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)

# Dilate to enlarge corner markers for visibility
harris = cv2.dilate(harris, None)

# Mark corners in red where response > 1% of max
img[harris > 0.01 * harris.max()] = [0, 0, 255]
```

The Feature Matching Pipeline

Given keypoints in two images, the classical pipeline for establishing correspondences (e.g., for panorama stitching, 3D reconstruction, or object recognition) consists of three stages:

1. **Detection:** Identify repeatable interest points (corners, blobs) in each image.
2. **Description:** For each keypoint, compute a descriptor vector encoding the local neighborhood. The descriptor must be robust to rotation, illumination changes, and ideally scale.
3. **Matching:** Compare descriptors between images using a distance metric (Euclidean for continuous descriptors, Hamming for binary) and select the closest pairs as correspondences.

Advanced Feature Descriptors

Harris corners are rotation-invariant but **not scale-invariant**: zoom into a corner far enough and it becomes an edge. Modern classical descriptors address this limitation.

SIFT (Scale-Invariant Feature Transform)

SIFT achieves invariance to scale, rotation, and moderate affine distortion through a multi-stage pipeline:

Detection (DoG): Constructs a **scale space** by convolving the image with Gaussians at increasing σ. The **Difference of Gaussians (DoG)** $D(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma)$ approximates the scale-normalized Laplacian. Keypoints are local extrema in (x, y, σ) space—points that persist across scales.

Orientation Assignment: For each keypoint, a histogram of gradient orientations in its neighborhood determines the **dominant orientation**. All subsequent computations are relative to this orientation, ensuring rotation invariance.

Descriptor: A 16×16 patch around the keypoint is divided into a 4×4 grid of cells. Each cell contributes an 8-bin **Histogram of Oriented Gradients (HOG)**. Concatenation yields a $4 \times 4 \times 8 = \mathbf{128}$-dimensional continuous feature vector, normalized to unit length for illumination invariance.

```
import cv2

img = cv2.imread('building.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Create SIFT detector and compute keypoints + descriptors
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
# descriptors.shape = (N, 128) -- N keypoints, 128-dim each

# Draw keypoints with orientation and scale
output = cv2.drawKeypoints(
    gray, keypoints, img,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
```

ORB (Oriented FAST and Rotated BRIEF)

ORB was designed as an efficient, royalty-free alternative to SIFT, widely used in real-time robotics and mobile applications.

FAST Detection: Identifies keypoints by comparing a center pixel's intensity to a ring of 16 surrounding pixels. If N contiguous pixels on the ring are all significantly brighter (or all darker) than the center, it is classified as a corner. FAST is extremely efficient but lacks orientation information.

Orientation ("Oriented"): ORB adds rotation invariance by computing the **intensity centroid** of the patch and defining orientation as the direction from the corner to the centroid.

BRIEF Description: Generates a **binary descriptor** (256 bits by default) by performing pairwise intensity comparisons at pre-selected pixel locations in the neighborhood. Each comparison yields one bit: 1 if $I(p_1) > I(p_2)$, else 0.

Matching with Hamming Distance: Because descriptors are binary strings, matching reduces to XOR followed by popcount—executed in a single CPU instruction. This is orders of magnitude faster than Euclidean distance on 128-dim floats.

```
import cv2

img = cv2.imread('building.jpg', cv2.IMREAD_GRAYSCALE)

# Create ORB detector (default: 500 keypoints, 256-bit descriptors)
orb = cv2.ORB_create(nfeatures=1000)
keypoints, descriptors = orb.detectAndCompute(img, None)
# descriptors.shape = (N, 32) -- 32 bytes = 256 bits per keypoint
```

Feature Matching in Practice

Given descriptors from two images, a **Brute-Force Matcher** compares every descriptor in image A against every descriptor in image B:

```
import cv2

img1 = cv2.imread('scene1.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('scene2.jpg', cv2.IMREAD_GRAYSCALE)

# --- SIFT matching (L2 norm on 128-dim float vectors) ---
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)  # L2 for SIFT
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)  # best first

# --- ORB matching (Hamming distance on binary descriptors) ---
orb = cv2.ORB_create(nfeatures=1000)
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)  # Hamming for ORB
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)
```

Remark 46 (SIFT vs. ORB Trade-off)**.** SIFT produces highly discriminative 128-dim float descriptors matched with L_2 distance—accurate but slow ($O(128n^2)$ for brute-force). ORB produces compact 256-bit binary descriptors matched with Hamming distance via hardware XOR—roughly 10–100× faster. For real-time systems (SLAM, AR), ORB is preferred; for offline tasks requiring maximum accuracy (3D reconstruction, archival matching), SIFT remains the gold standard among classical methods.

Feature Engineering for Images (Part 3)

Combining Filters: Unsharp Masking

Filters are linear operations, which means they possess the property of superposition and can be algebraically combined. A classic example is the **Sharpening Filter**, which can be derived from a combination of the original image and a smoothing filter.

This process is commonly known as **Unsharp Masking**. It can be broken down into two logical steps:

1. **Extract Details (High-Pass):** Take a smoothed (blurred) version of the image and subtract it from the original image. This removes the low-frequency information (smooth gradients, general color) and leaves only the high-frequency *details* (edges and noise).

$$I_{\text{details}} = I_{\text{original}} - I_{\text{smoothed}}$$

2. **Enhance Image:** Add these extracted details back to the original image, scaled by a factor α, to emphasize the edges.

$$I_{\text{sharp}} = I_{\text{original}} + \alpha \cdot I_{\text{details}}$$

Remark 47 (Linearity of Filter Operations)**.** Because convolution is a linear operation, we can combine the two steps into a single filter kernel. Substituting the first equation into the second: $I_{\text{sharp}} = (1+\alpha) \cdot I_{\text{original}} - \alpha \cdot I_{\text{smoothed}}$. This means a single convolution pass can achieve the sharpening effect, which is computationally efficient.

```
import cv2
import numpy as np

img = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)
# Step 1: Smooth with Gaussian blur
smoothed = cv2.GaussianBlur(img, (7, 7), sigmaX=2)
# Step 2: Extract high-frequency details
details = img.astype(np.float32) - smoothed.astype(np.float32)
# Step 3: Add scaled details back (alpha controls strength)
alpha = 1.5
sharpened = np.clip(img + alpha * details, 0, 255).astype(np.uint8)
```

Non-Linear Filters: Median Filter

Not all useful filters are linear (convolutional). The **Median Filter** replaces each pixel with the *median* of pixel values in its local neighborhood window. There is no set of weights a_i such that $\text{median}(p_1, \ldots, p_n) = \sum a_i p_i$, so the median filter is fundamentally non-linear.

The median filter is particularly effective at removing **salt-and-pepper noise** (random black and white pixels) because the median statistic is robust to outliers—an extreme value (0 or 255) will not affect the median of a neighborhood dominated by normal values.

```
from skimage.filters import median
from skimage import io

noisy_img = io.imread('noisy_image.png', as_gray=True)
cleaned = median(noisy_img)  # Default disk-shaped footprint
```

Image Gradients

To understand shape-based features, we must first formalize the notion of an **image gradient**. For a grayscale image where $f(x, y)$ records the intensity at pixel (x, y), the gradient vector is:

$$\nabla f(x,y) = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} f(x+1,y) - f(x-1,y) \\ f(x,y+1) - f(x,y-1) \end{bmatrix}$$

Since images are discrete, the partial derivatives are approximated by finite differences between neighboring pixels. From this gradient vector we extract two key quantities:

Magnitude: $G = \sqrt{G_x^2 + G_y^2}$ — measures the *strength* of the intensity change (large at edges, near-zero in flat regions).

Direction: $\Phi = \arctan(G_y/G_x)$ — measures the *orientation* of the intensity change (perpendicular to the edge).

Remark 48. The x-gradient G_x "fires" on vertical edges (intensity changes left-to-right), while the y-gradient G_y fires on horizontal edges. The magnitude image highlights all edges regardless of orientation.

Histogram of Oriented Gradients (HOG)

HOG (Histogram of Oriented Gradients) is a feature descriptor that captures the distribution of gradient directions in localized portions of an image. Unlike simple edge detection, which only asks "is this pixel an edge?", HOG encodes *how edges are oriented*—providing rich structural and shape information.

HOG was introduced by Dalal and Triggs (2005) for pedestrian detection and remains widely used for object detection and classification tasks (pedestrians, faces, vehicles).

The HOG Pipeline (Step by Step)

1. **Preprocess:** Resize the image to a standard size with dimensions that divide evenly into cells (e.g., 64×128, using powers of 2).
2. **Compute Gradients:** For every pixel, compute G_x, G_y, and then the magnitude G and orientation Φ.
3. **Divide into Cells:** Partition the image into small spatial cells (e.g., 8×8 pixels). Each cell will produce one local histogram.
4. **Build Orientation Histograms:** Within each cell, create a histogram of gradient orientations using B bins over $[0°, 180°]$ (unsigned gradients). Crucially, each pixel *votes* for its orientation bin weighted by its gradient magnitude (not just a count of 1). To reduce bin-edge artifacts, the magnitude contribution is interpolated between the two nearest bins proportionally.
5. **Normalize Across Blocks:** Group adjacent cells into larger overlapping blocks (e.g., 2×2 cells $= 16 \times 16$ pixels). Normalize the concatenated histograms within each block using L2-norm:

   $$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\sqrt{\|\mathbf{v}\|^2 + \epsilon}}$$

   This provides invariance to local illumination changes and contrast variations.
6. **Concatenate:** The normalized block histograms from the entire image are concatenated into a single, fixed-length feature vector—the HOG descriptor.

Remark 49 (Why Magnitude-Weighted Votes?)**.** Using gradient magnitude as the vote weight ensures that strong, confident edges contribute more to the histogram than weak, noisy gradients. This makes the descriptor robust to noise in flat regions.

Remark 50 (Why Block Normalization?)**.** Image gradients are sensitive to overall lighting. A bright region will have larger gradient magnitudes than a dark region showing the same structure. Normalizing over overlapping blocks compensates for this, making the descriptor robust to illumination variation.

HOG in Practice with scikit-image

```
from skimage.feature import hog
from skimage import io
from skimage.color import rgb2gray
```

```python
img = io.imread('cat.png')
img_gray = rgb2gray(img)

# Extract HOG features
features, hog_image = hog(
    img_gray,
    orientations=8,          # Number of orientation bins
    pixels_per_cell=(8, 8),  # Cell size in pixels
    cells_per_block=(3, 3),  # Block size in cells
    visualize=True,          # Also return HOG visualization
    block_norm='L2-Hys'      # Block normalization method
)
print(f"HOG feature vector length: {features.shape[0]}")
```

HOG Feature Vector Dimensionality

The length of the HOG descriptor depends on the image size and parameters. For an image of size $H \times W$ with cell size $c \times c$, block size $b \times b$ cells, and B orientation bins:

$$\dim = \left(\left\lfloor \frac{H/c - b}{1} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{W/c - b}{1} \right\rfloor + 1 \right) \times b^2 \times B$$

where the block stride is typically 1 cell (blocks overlap). This produces a fixed-length vector regardless of image content, making it directly usable as input to any classical ML model.

Effect of Cell Size

Smaller cells (e.g., 8×8) capture fine-grained structural detail but produce longer feature vectors. Larger cells (e.g., 16×16) are more compact and capture coarser shape information. The choice depends on the scale of the objects of interest.

```python
# Compare different cell sizes
feat_8, _ = hog(img_gray, orientations=8,
                pixels_per_cell=(8, 8),
                cells_per_block=(3, 3), visualize=True)
feat_16, _ = hog(img_gray, orientations=8,
                 pixels_per_cell=(16, 16),
                 cells_per_block=(3, 3), visualize=True)
print(f"8x8 cells: {feat_8.shape[0]} features")
print(f"16x16 cells: {feat_16.shape[0]} features")
```

Template Matching

Template Matching is a brute-force technique for finding a small reference image (the *template*) within a larger image. The template is slid across every possible position in the search image, and at each location a **similarity score** is computed between the template and the underlying patch.

Methods

Cross-Correlation: Compute the dot product between the template T and the image patch I at each position (x, y):

$$R(x, y) = \sum_{x', y'} T(x', y') \cdot I(x + x', y + y')$$

The position with the highest R indicates the best match. However, raw cross-correlation is biased toward bright regions.

Normalized Cross-Correlation (NCC): Divides by the energy of both the template and the patch, producing scores in $[-1, 1]$. This is robust to global brightness and contrast differences.

Remark 51 (Limitations)**.** Template matching is **not scale-invariant** and **not rotation-invariant**: if the object in the search image is a different size or rotated relative to the template, matching will fail. For scale/rotation-robust matching, use feature-based methods (SIFT, ORB) from Part 2.

```
import cv2
import numpy as np

img = cv2.imread('scene.jpg', cv2.IMREAD_GRAYSCALE)
template = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)

# Normalized cross-correlation matching
result = cv2.matchTemplate(img, template, cv2.TM_CCOEFF_NORMED)

# Find best match location
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
top_left = max_loc  # Top-left corner of best match
h, w = template.shape
bottom_right = (top_left[0] + w, top_left[1] + h)
print(f"Best match at {top_left}, score={max_val:.3f}")
```

Image Preprocessing Pipelines

Real-world image classification rarely uses a single feature type. A typical **preprocessing pipeline** chains multiple operations to transform raw pixels into a clean, informative feature vector suitable for ML models.

Common Pipeline Structure

1. **Resize:** Standardize all images to the same dimensions (required for fixed-length feature vectors).
2. **Grayscale Conversion:** Reduce from 3 channels to 1 when color is not informative, cutting feature dimensionality by $3\times$.
3. **Denoise:** Apply a smoothing or median filter to suppress sensor noise before feature extraction.
4. **Feature Extraction:** Compute one or more descriptors (HOG, edge histograms, pixel statistics, etc.).
5. **Normalize:** Scale the feature vector (e.g., zero mean, unit variance) so that no single feature dominates due to scale differences.

Building a Combined Feature Vector

Different descriptors capture complementary information. We can concatenate multiple feature vectors into a single, richer representation:

```
import numpy as np
from skimage.feature import hog
from skimage.color import rgb2gray
from skimage import io
from skimage.transform import resize

def extract_features(image_path):
    img = io.imread(image_path)
    img = resize(img, (128, 64), anti_aliasing=True)
    gray = rgb2gray(img)

    # Feature 1: HOG descriptor (shape information)
    hog_feat = hog(gray, orientations=9,
                   pixels_per_cell=(8, 8),
                   cells_per_block=(2, 2))

    # Feature 2: Basic pixel statistics
    stats = np.array([gray.mean(), gray.std(),
                      gray.min(), gray.max()])
```

```
    # Concatenate into one feature vector
    return np.concatenate([hog_feat, stats])
```

Putting It All Together: Image Classification with Classical ML

With a complete feature extraction pipeline, we can train any `sklearn` classifier on image data. The key insight: **traditional ML models require flat, fixed-length feature vectors**—they cannot operate on raw 2D pixel arrays of varying sizes. The entire pipeline above exists to solve this representation problem.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

# Assume image_paths and labels are defined
X = np.array([extract_features(p) for p in image_paths])
y = np.array(labels)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Normalize features (critical for SVM)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train classifier
clf = SVC(kernel='rbf', C=10)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

Remark 52 (Feature Normalization is Critical)**.** Different feature types live on different scales (e.g., HOG values are small floats, pixel statistics may range 0–255). Without normalization via `StandardScaler`, distance-based models (SVM, KNN) will be dominated by whichever feature has the largest numerical range, regardless of its actual discriminative power.

Remark 53 (Classical Pipeline vs. Deep Learning)**.** This entire manual pipeline—resize, denoise, hand-craft features, normalize, classify—is what **Convolutional Neural Networks (CNNs)** learn to replace end-to-end. CNNs learn the feature extraction filters directly from data via backpropagation. However, when labeled data is scarce (< 1000 images), these classical pipelines with hand-crafted features (especially HOG + SVM) often outperform deep learning.

Linear Models: Definitions, Properties, and Training

Linear Model Definition

Linear models hypothesize that the target output is a weighted linear combination of input features.

Definition 46 (Linear Model)**.** For input $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T \in \mathbb{R}^d$, the linear model predicts:

$$f_{\mathbf{w}}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + w_0 = \mathbf{w}^T \mathbf{x} + w_0$$

where $\mathbf{w} = [w_1, \ldots, w_d]^T$ is the **weight vector** and w_0 is the **bias** (intercept).

Remark 54 (Extended Feature Vector)**.** To absorb the bias into the dot product, augment the input with a constant 1:

$$\mathbf{x}' = [1, x_1, \ldots, x_d]^T, \quad \mathbf{w}' = [w_0, w_1, \ldots, w_d]^T$$
$$f_{\mathbf{w}}(\mathbf{x}) = (\mathbf{w}')^T \mathbf{x}'$$

This compact form treats all parameters uniformly—no special handling for w_0.

```
# Linear regression with sklearn
from sklearn.linear_model import LinearRegression
import numpy as np

model = LinearRegression()          # f(x) = w^T x + w_0
model.fit(X_train, y_train)         # Learn optimal weights
print(f"w = {model.coef_}")         # Weight vector
print(f"w_0 = {model.intercept_}")  # Bias term
y_pred = model.predict(X_test)      # Predict on new data
```

Classification vs. Regression

The raw output $f_{\mathbf{w}}(\mathbf{x}) \in \mathbb{R}$ is continuous. Its interpretation depends on the task:

Regression: The prediction is the raw output: $\hat{y} = f_{\mathbf{w}}(\mathbf{x})$. The model fits a linear curve to continuous targets.

Binary Classification: The prediction is the **sign** of the output: $\hat{y} = \text{sgn}(f_{\mathbf{w}}(\mathbf{x})) \in \{-1, +1\}$. The model defines a linear decision boundary (hyperplane) separating classes.

Geometrically, $\mathbf{w}$ is orthogonal to the decision hyperplane $f_{\mathbf{w}}(\mathbf{x}) = 0$, and the bias w_0 shifts it away from the origin. Points on one side yield $f > 0$ (class $+1$), and on the other $f < 0$ (class -1).

```
# Binary classification with sign of linear output
import numpy as np

w = np.array([0.5, -1.0])    # Weight vector
b = -2.0                      # Bias
x = np.array([4.0, 2.0])     # Test input
score = w @ x + b             # f_w(x) = 0.5*4 + (-1)*2 - 2 = -2
y_hat = np.sign(score)        # sgn(-2) = -1 -> class -1
```

Score and Functional Margin

Definition 47 (Functional Margin)**.** For binary classification with $y \in \{-1, +1\}$, the **functional margin** of example $(\mathbf{x}, y)$ is:

$$\mu(\mathbf{x}) = y \cdot f_{\mathbf{w}}(\mathbf{x})$$

- $\mu > 0$: prediction is **correct** (score sign matches true label).
- $\mu < 0$: prediction is **incorrect**.
- $|\mu|$: the **confidence**—how far $\mathbf{x}$ lies from the decision boundary.

The **geometric margin** normalizes by $\|\mathbf{w}\|$, giving the true Euclidean distance to the boundary:

$$\text{geometric margin}(\mathbf{x}) = \frac{\mu(\mathbf{x})}{\|\mathbf{w}\|}$$

Non-Linear Feature Transformations

Raw features may not be linearly separable. The key insight: map inputs to a higher-dimensional space where linear separation becomes possible.

Definition 48 (Feature Transformation)**.** A **feature map** $\Phi : \mathbb{R}^d \to \mathbb{R}^m$ transforms the input into m basis functions $\phi_1(\mathbf{x}), \ldots, \phi_m(\mathbf{x})$. The model becomes:

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}) = \sum_{j=1}^{m} w_j \phi_j(\mathbf{x}) + w_0$$

This is **linear in the transformed space** $\mathbb{R}^m$ but **non-linear in the original space** $\mathbb{R}^d$.

Remark 55 (Polynomial Features)**.** For $\mathbf{x} = [x_1, x_2]^T$, a degree-2 polynomial transformation produces:

$$\Phi(\mathbf{x}) = [1,\ x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2]^T$$

More generally, polynomial basis functions of degree p with d-dimensional input yield $\binom{d+p}{p}$ features. Higher degrees give more expressive power but risk overfitting.

Remark 56 (Basis Functions)**.** Common choices include polynomial bases ($\phi_k(x) = x^k$), Fourier bases ($\sin(kx), \cos(kx)$), and radial basis functions. In principle, a complete basis can approximate any function to arbitrary precision, but this may require infinitely many features—motivating the kernel trick (covered later).

```
# Polynomial feature transformation
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline

X = np.array([[1], [2], [3], [4]])  # 1D input
y = np.array([1, 4, 9, 16])         # y = x^2

# Degree-2 poly: [1, x, x^2] -> linear in transformed space
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),     # phi(x) = [1, x, x^2]
    LinearRegression()                # w^T phi(x)
)
poly_model.fit(X, y)
```

Training: Empirical Risk Minimization

Training finds weights $\mathbf{w}^*$ minimizing average loss over N training examples.

Definition 49 (ERM for Linear Models)**.**

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\mathbf{w}}(\mathbf{x}^{(i)}),\ y^{(i)}\Big)$$

where $\ell(\hat{y}, y)$ is the **loss function**. The choice of ℓ determines the solution's properties and the difficulty of the optimization problem.

Loss Functions for Regression

For continuous targets $y \in \mathbb{R}$:

Squared Loss (L_2): $\ell(\hat{y}, y) = (y - \hat{y})^2$. Strongly penalizes outliers. Yields Ordinary Least Squares (OLS). Differentiable everywhere.

Absolute Loss (L_1): $\ell(\hat{y}, y) = |y - \hat{y}|$. Robust to outliers. Non-differentiable at $\hat{y} = y$.

Loss Functions for Classification

For binary labels $y \in \{-1, +1\}$, losses are defined in terms of the margin $\mu = y \cdot f_{\mathbf{w}}(\mathbf{x})$:

0–1 Loss: $\ell(\mu) = \mathbf{1}[\mu < 0]$. Ideal metric but **non-convex** and non-differentiable—computationally intractable.

Hinge Loss: $\ell(\mu) = \max(0, 1 - \mu)$. Convex surrogate for 0–1 loss. Used in **Support Vector Machines**. Penalizes even correct predictions with margin < 1.

Logistic Loss: $\ell(\mu) = \log(1 + e^{-\mu})$. Smooth, differentiable, convex. Used in **Logistic Regression**. Never fully "saturates"—always provides a gradient signal.

Exponential Loss: $\ell(\mu) = e^{-\mu}$. Aggressive penalty on misclassifications. Used in **AdaBoost**. Highly sensitive to outliers.

Remark 57. All surrogate losses (hinge, logistic, exponential) upper-bound the 0–1 loss. They are convex, making optimization tractable, and each induces different properties: hinge creates sparse support vectors, logistic outputs calibrated probabilities, and exponential drives boosting algorithms.

Regularization

High-dimensional feature spaces (e.g., degree-p polynomials) risk overfitting. **Regularization** adds a penalty $R(\mathbf{w})$ to shrink or sparsify weights:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \left[\frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\mathbf{w}}(\mathbf{x}^{(i)}), y^{(i)}\Big) + \lambda\, R(\mathbf{w}) \right]$$

where $\lambda > 0$ controls regularization strength.

L_2 **Regularization (Ridge):** $R(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_j w_j^2$. Shrinks all weights toward zero. Produces smoother models. Closed-form solution exists.

L_1 **Regularization (Lasso):** $R(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_j |w_j|$. Drives irrelevant weights **exactly to zero**, performing automatic **feature selection**. No closed-form; requires iterative optimization.

```
# Ridge (L2) vs Lasso (L1) regularization
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)   # L2: shrinks all weights
ridge.fit(X_train, y_train)
print(f"Ridge weights: {ridge.coef_}")  # Small but nonzero

lasso = Lasso(alpha=0.1)   # L1: drives some weights to 0
lasso.fit(X_train, y_train)
print(f"Lasso weights: {lasso.coef_}")  # Sparse: some exactly 0
print(f"Nonzero features: {np.sum(lasso.coef_ != 0)}")
```

Optimization: Finding Optimal Weights

The ERM objective $L(\mathbf{w})$ must be minimized. Two approaches:

1. **Analytical (Closed-Form) Solution:** Set $\nabla_{\mathbf{w}} L(\mathbf{w}) = \mathbf{0}$ and solve. Possible for squared loss (OLS) and Ridge regression. For OLS with design matrix X:

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$$

   For Ridge: $\mathbf{w}^* = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$. Cost is $O(d^3)$ for the matrix inverse.

2. **Gradient Descent (Iterative):** When no closed-form exists (logistic, hinge, exponential losses), iteratively follow the negative gradient:

Definition 50 (Gradient Descent Update)**.**

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \nabla_{\mathbf{w}} L\Big(\mathbf{w}^{(t)}\Big)$$

where $\eta > 0$ is the **learning rate**. Each step moves along the steepest descent direction of the loss surface. Convergence requires η to be small enough (for convex losses, gradient descent converges to the global minimum).

Remark 58. Computing $\nabla_{\mathbf{w}} L(\mathbf{w})$ requires summing over all N training examples per iteration. **Stochastic Gradient Descent** (SGD) approximates the full gradient using a single example (or mini-batch), trading per-step accuracy for computational efficiency.

Gradient Methods for Model Optimization

The Optimization Problem

Once a model hypothesis $f_{\mathbf{w}}(\mathbf{x})$ and a loss function $\ell(\hat{y}, y)$ are chosen, the machine learning task reduces to **Empirical Risk Minimization (ERM)**: find the parameter vector $\mathbf{w}^*$ that minimizes average loss over the training set:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} L(\mathbf{w}) = \arg\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_{\mathbf{w}}(\mathbf{x}^{(i)}),\ y^{(i)}\big)$$

Some linear models (e.g., OLS) admit a closed-form solution via $\nabla_{\mathbf{w}} L = \mathbf{0}$. Most models—logistic regression, neural networks, SVMs with kernels—yield loss surfaces with no analytical solution, requiring **iterative numerical optimization**.

Gradient Descent Fundamentals

Definition 51 (Gradient Descent (GD))**.** Gradient Descent is an iterative first-order optimization algorithm. At each step it evaluates the gradient $\nabla_{\mathbf{w}} L(\mathbf{w})$ at the current point and moves in the direction of steepest descent.

The update rule at iteration t:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

Below is a from-scratch implementation on the convex quadratic $f(x_1, x_2) = \frac{1}{2}x_1^2 + \frac{5}{2}x_2^2$, whose gradient is $\nabla f = (x_1,\ 5x_2)^T$ and global minimum is at the origin:

```
import numpy as np

# Define a simple convex quadratic and its gradient
def f(x):
    return 0.5 * x[0]**2 + 2.5 * x[1]**2

def grad_f(x):
    return np.array([x[0], 5 * x[1]])

# Gradient descent from scratch
def gradient_descent(grad, x0, lr=0.1, n_iters=50, tol=1e-6):
    x = x0.copy()
    history = [x.copy()]
    for _ in range(n_iters):
        g = grad(x)
        x = x - lr * g                # GD update rule
        history.append(x.copy())
        if np.linalg.norm(g) < tol:   # stop if gradient ~0
            break
    return x, history

x_opt, path = gradient_descent(grad_f, np.array([2.0, 0.4]))
print(f"Minimum at: {x_opt}")         # -> ~[0, 0]
```

The Learning Rate (η)

The **learning rate** (step size) η controls how far each update moves along the negative gradient. Even with fixed η, the actual step length in parameter space is $\|\mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}\| = \eta \cdot \|\nabla L(\mathbf{w}^{(t)})\|$, so steps shrink naturally near a minimum where gradients vanish.

- **Too small** η: convergence is painfully slow; the optimizer may stall in sub-optimal regions.
- **Too large** η: updates overshoot the minimum, causing oscillation or divergence.

The following snippet demonstrates both failure modes on the same quadratic:

```
import numpy as np

def grad_f(x):
    return np.array([x[0], 5 * x[1]])

x0 = np.array([2.0, 0.4])

# Too-small learning rate: barely moves in 200 steps
x = x0.copy()
for _ in range(200):
    x = x - 0.001 * grad_f(x)
print(f"lr=0.001 after 200 steps: {x}")   # still far from [0,0]

# Too-large learning rate: diverges!
x = x0.copy()
for _ in range(10):
    x = x - 0.5 * grad_f(x)
print(f"lr=0.5  after 10 steps:  {x}")    # explodes

# Good learning rate: converges smoothly
x = x0.copy()
for _ in range(50):
    x = x - 0.1 * grad_f(x)
print(f"lr=0.1  after 50 steps:  {x}")    # ~[0, 0]
```

Remark 59 (Convexity vs. Non-Convexity)**.** If $L(\mathbf{w})$ is **convex** (e.g., linear/logistic regression losses), GD converges to the unique global minimum. If L is **non-convex** (e.g., neural networks), the algorithm may get trapped in *local minima*, *saddle points*, or flat *plateaus*. In practice, SGD noise and adaptive methods help escape shallow traps.

Data-Sampling Variants

Computing ∇L requires summing over training examples. How many examples per update defines three variants:

Batch Gradient Descent (BGD): Uses the **entire** dataset (N samples) per update.

- *Pros:* Accurate gradient estimate; stable, predictable convergence.
- *Cons:* Computationally expensive per step; impractical for large N.

Stochastic Gradient Descent (SGD): Uses **one randomly selected** example per update.

- *Pros:* Extremely fast per iteration; low memory. The noise can help escape shallow local minima.
- *Cons:* High variance causes the loss to fluctuate wildly; may never settle precisely at the minimum.

Mini-Batch Gradient Descent: The standard compromise—uses a random subset of B examples (e.g., $B \in \{32, 64, 128\}$).

- *Pros:* Reduces update variance vs. SGD; exploits GPU vectorization, making it faster than BGD in wall-clock time.

Remark 60 (Terminology)**.** In modern deep learning, "SGD" almost always refers to mini-batch GD. Pure single-sample SGD is rarely used in practice.

The conceptual difference in code: BGD computes gradients over all N rows; SGD over 1; mini-batch over a random slice of size B:

```
import numpy as np

# Assume X (N x d), y (N,), w (d,) for linear regression
# MSE gradient: (1/m) * X^T (Xw - y)

def grad_batch(X, y, w):
    """Full-batch gradient (all N samples)."""
    r = X @ w - y
```

```
    return X.T @ r / len(y)

def grad_sgd(X, y, w):
    """Stochastic gradient (1 random sample)."""
    i = np.random.randint(len(y))
    r = X[i] @ w - y[i]
    return X[i] * r                # single-sample gradient

def grad_minibatch(X, y, w, B=32):
    """Mini-batch gradient (B random samples)."""
    idx = np.random.choice(len(y), B, replace=False)
    r = X[idx] @ w - y[idx]
    return X[idx].T @ r / B
```

Advanced Gradient Methods

Standard GD can struggle with pathological curvature (e.g., long narrow ravines) and sensitivity to learning rate choice. Advanced optimizers modify the update rule to accelerate and stabilize convergence.

Momentum-Based Methods

Definition 52 (Momentum)**.** Instead of relying solely on the current gradient, Momentum accumulates an exponentially decaying moving average of past gradients. It acts like a ball rolling downhill—gaining speed in consistent directions and dampening oscillations in erratic ones.

Standard Momentum: Maintains a velocity vector:

$$v_t = \gamma\, v_{t-1} + \eta\, \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - v_t$$

The decay factor $\gamma \in [0, 1)$ (typically 0.9) controls how much past gradients influence the current update.

Nesterov Accelerated Gradient (NAG): A "look-ahead" variant. Instead of computing the gradient at $\mathbf{w}^{(t)}$, NAG evaluates it at the predicted future position $\mathbf{w}^{(t)} - \gamma\, v_{t-1}$. This anticipatory correction reduces overshooting, especially in regions of steep curvature.

Adaptive Learning Rate Methods

Rather than one global η for all parameters, adaptive methods tune the effective learning rate per parameter w_j.

AdaGrad: Scales each parameter's learning rate inversely by the square root of the cumulative sum of its historical squared gradients. Larger updates for infrequent features, smaller for frequent ones. *Drawback:* the accumulated sum grows monotonically, eventually shrinking the learning rate to zero and halting training.

RMSprop: Fixes AdaGrad's decay problem by replacing the cumulative sum with an **exponentially decaying average** of squared gradients, keeping the effective learning rate from vanishing.

Adam (Adaptive Moment Estimation)

Definition 53 (Adam Optimizer)**.** Adam is the most widely used default optimizer in deep learning. It combines **Momentum** (1st moment—gradient direction) with **RMSprop** (2nd moment—gradient magnitude scaling).

The Adam update at iteration t consists of four steps:

1. **1st Moment (Momentum):** Exponentially weighted average of past gradients.

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, \nabla f(\mathbf{w}_{t-1})$$

2. **2nd Moment (Adaptive Scaling):** Exponentially weighted average of past squared gradients.

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, \big(\nabla f(\mathbf{w}_{t-1})\big)^2$$

3. **Bias Correction:** Since $m_0 = v_0 = 0$, early estimates are biased toward zero. Correct:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

4. **Parameter Update:**

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\,\hat{m}_t$$

   where $\epsilon \approx 10^{-8}$ prevents division by zero.

Remark 61 (Why Adam Works)**.** Adam's effective step size is fully adaptive. Large gradient magnitudes shrink the step (preventing divergence); small gradients in plateaus enlarge it (preventing stalling). Combined with momentum's directional smoothing, Adam achieves robust convergence across diverse non-convex landscapes. Default hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$) work well in most settings.

Gradient Methods in Practice with `scikit-learn`

`scikit-learn` exposes SGD-based linear models via `SGDClassifier` and `SGDRegressor`. These use mini-batch SGD under the hood and accept key hyperparameters: learning rate schedule, regularization, and number of passes over the data.

```
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_regression

# Generate synthetic regression data
X, y = make_regression(n_samples=500, n_features=5, noise=10,
                       random_state=42)

# SGD is sensitive to feature scale -- always standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# SGDRegressor uses squared loss by default (= linear regression)
sgd_reg = SGDRegressor(
    loss='squared_error',
    learning_rate='invscaling',   # eta = eta0 / t^power_t
    eta0=0.01,                    # initial learning rate
    max_iter=1000,
    tol=1e-4,
    random_state=42
)
sgd_reg.fit(X_scaled, y)
print(f"R^2 = {sgd_reg.score(X_scaled, y):.4f}")
print(f"Learned weights: {sgd_reg.coef_}")
```

```
from sklearn.linear_model import SGDClassifier
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=1000, n_features=10,
                           random_state=42)
X = StandardScaler().fit_transform(X)

# SGDClassifier with log_loss = logistic regression via SGD
clf = SGDClassifier(
    loss='log_loss',              # logistic regression
    learning_rate='optimal',      # auto-tuned schedule
    max_iter=1000,
    random_state=42
)
clf.fit(X, y)
```

```python
print(f"Accuracy: {clf.score(X, y):.4f}")
```

Support Vector Machines (SVM)

The Optimal Decision Boundary

For linearly separable data, infinitely many hyperplanes can perfectly separate two classes. **Support Vector Machines (SVMs)** answer the question: *which of these boundaries generalizes best?*

The SVM principle is intuitive: the best decision boundary is the one **furthest from the closest training points** of any class. A classifier with a wider "safe zone" around its boundary is less likely to misclassify unseen data that falls near the border. This safe zone is formalized as the **geometric margin**.

Definition 54 (Geometric Margin)**.** The **margin** γ is the perpendicular distance between the decision hyperplane $\mathbf{w}^T\mathbf{x}+b=0$ and the closest data points from each class. The total margin width is $\gamma = \frac{2}{\|\mathbf{w}\|}$. SVM seeks the hyperplane that **maximizes** γ.

Definition 55 (Support Vectors)**.** The **support vectors** are the data points lying exactly on the margin boundaries (i.e., satisfying $y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) = 1$). The optimal hyperplane depends *only* on these critical points—removing or moving any non-support-vector training point leaves the decision boundary unchanged.

```
# Train a linear SVM classifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

svm_clf = SVC(kernel='linear')       # Linear (hard/soft) SVM
svm_clf.fit(X_train, y_train)
y_pred = svm_clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

```
# Inspect support vectors after fitting
print(f"Support vectors per class: {svm_clf.n_support_}")
print(f"Total support vectors: {len(svm_clf.support_vectors_)}")

# Access the learned hyperplane: w^T x + b = 0
print(f"Weights w: {svm_clf.coef_[0]}")
print(f"Intercept b: {svm_clf.intercept_[0]:.4f}")
```

Hard Margin SVM

Hard Margin SVM enforces that every training point is correctly classified and lies outside the margin band. Since the margin width is $\frac{2}{\|\mathbf{w}\|}$, maximizing the margin is equivalent to minimizing $\|\mathbf{w}\|^2$. The resulting **quadratic program** (convex, with linear inequality constraints) is:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \qquad \text{s.t.} \quad y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \ldots, m$$

Remark 62 (Why $\frac{1}{2}\|\mathbf{w}\|^2$?)**.** The geometric margin $\gamma = \frac{2c}{\|\mathbf{w}\|}$ is scaling-invariant in $\mathbf{w}$, so we can fix $c = 1$ without loss of generality. Squaring removes the square root (a monotonic transformation that preserves the optimum) and the $\frac{1}{2}$ simplifies the gradient to $\mathbf{w}$.

Remark 63. Hard Margin SVM requires **perfect linear separability**. A single outlier crossing the boundary can make the problem infeasible or drastically skew the solution.

Primal and Dual Formulations

The **primal** solves for the d-dimensional weight vector $\mathbf{w}$. Introducing Lagrange multipliers $\alpha_i \geq 0$ for each constraint yields the equivalent **dual** problem over m variables:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j \, y^{(i)} y^{(j)} \, (\mathbf{x}^{(i)})^T \mathbf{x}^{(j)} \qquad \text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y^{(i)} = 0, \;\; \alpha_i \geq 0$$

At the solution, $\mathbf{w} = \sum_{i=1}^{m} \alpha_i y^{(i)} \mathbf{x}^{(i)}$. Only support vectors have $\alpha_i > 0$; all other training points have $\alpha_i = 0$ and are irrelevant. The bias is recovered as $b = y^{(k)} - \mathbf{w}^T \mathbf{x}^{(k)}$ for any support vector k.

Soft Margin SVM

When data is not perfectly separable (or contains noisy outliers), we relax the hard constraints by introducing **slack variables** $\xi_i \geq 0$ that allow individual points to violate the margin or be misclassified:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{m} \xi_i \qquad \text{s.t.} \quad y^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0$$

The slack variable ξ_i has a direct geometric meaning:

- $\xi_i = 0$: point $\mathbf{x}^{(i)}$ is correctly classified and outside the margin.
- $0 < \xi_i < 1$: correctly classified but inside the margin (margin violation).
- $\xi_i \geq 1$: misclassified.

The C Parameter: C controls the trade-off between margin width and constraint violations.

- **Large C:** High penalty for violations $\Rightarrow$ narrower margin, fewer errors on training data (risk of overfitting).
- **Small C:** Low penalty for violations $\Rightarrow$ wider margin, more tolerance for misclassifications (better generalization, risk of underfitting).

As $C \to \infty$, the soft-margin formulation recovers the hard-margin SVM.

The soft-margin dual differs from the hard-margin dual only by an upper bound on each multiplier: $0 \leq \alpha_i \leq C$.

```
# Effect of C: small C = wide margin, large C = narrow margin
from sklearn.svm import SVC

svm_small_C = SVC(kernel='linear', C=0.01)  # Wide margin
svm_large_C = SVC(kernel='linear', C=100)   # Narrow margin

svm_small_C.fit(X_train, y_train)
svm_large_C.fit(X_train, y_train)

# Small C: more support vectors, wider margin
print(f"C=0.01 SVs: {sum(svm_small_C.n_support_)}")
# Large C: fewer support vectors, tighter fit
print(f"C=100  SVs: {sum(svm_large_C.n_support_)}")
```

SVM and the Hinge Loss

Soft Margin SVM can be understood through the **Empirical Risk Minimization (ERM)** framework. Training a soft-margin SVM is mathematically equivalent to minimizing the **hinge loss** with L_2 regularization:

$$\min_{\mathbf{w}} \sum_{i=1}^{m} \max\big(0, \; 1 - y^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b)\big) + \lambda \|\mathbf{w}\|^2$$

The equivalence follows by identifying the slack variables as $\xi_i = \max(0, 1 - y^{(i)} f_{\mathbf{w}}(\mathbf{x}^{(i)}))$ and the regularization strength as $\lambda = \frac{1}{2C}$.

Remark 64 (Hinge Loss vs. 0–1 Loss)**.** The hinge loss is a convex upper bound on the 0–1 loss. It penalizes not only misclassifications but also correct predictions that fall inside the margin ($y^{(i)} f_{\mathbf{w}}(\mathbf{x}^{(i)}) < 1$), pushing the algorithm to achieve a margin of at least 1 on every point. Keeping $\|\mathbf{w}\|$ small (via regularization) translates directly to a larger geometric margin, since $\gamma = \frac{2}{\|\mathbf{w}\|}$.

Remark 65 (SVM vs. Logistic Regression)**.** Both are linear classifiers, but they differ in their loss functions. Logistic regression uses the **log loss** (cross-entropy), which always imposes a penalty—even for correctly classified points far from the boundary. The hinge loss imposes **zero penalty** once a point is outside the margin ($y \cdot f(\mathbf{x}) \geq 1$), making SVMs sparser: only support vectors influence the solution.

The Kernel Trick

When data is highly non-linear, no hyperplane in the original feature space can achieve good separation. We could map inputs to a higher-dimensional space $\phi(\mathbf{x})$ where the data becomes separable, but explicitly computing $\phi(\mathbf{x})$ is often prohibitively expensive.

Definition 56 (The Kernel Trick)**.** In the dual formulation, training data appears only through dot products $(\mathbf{x}^{(i)})^T \mathbf{x}^{(j)}$. A **kernel function** $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$ computes the inner product in the transformed space *without ever explicitly computing* ϕ. Replacing dot products with K in the dual lets SVM learn non-linear boundaries at the cost of a linear kernel evaluation.

The prediction for a new input $\mathbf{z}$ becomes:

$$y(\mathbf{z}) = \operatorname{sign}\left(b + \sum_{i \in SV} \alpha_i \, y^{(i)} \, K(\mathbf{x}^{(i)}, \mathbf{z}) \right)$$

Common kernel functions:

- **Linear:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$ — no transformation (standard SVM).
- **Polynomial:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$ — maps to polynomial feature space of degree d.
- **RBF / Gaussian:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\!\big(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\big)$ — maps to an *infinite*-dimensional space. The parameter γ controls how quickly similarity decays with distance.

```python
# Kernel SVM with RBF (Gaussian) kernel
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# RBF kernel is sensitive to feature scale -- always standardize
rbf_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1.0, gamma='scale')  # RBF kernel
)
rbf_svm.fit(X_train, y_train)
print(f"RBF SVM accuracy: {rbf_svm.score(X_test, y_test):.3f}")

# Polynomial kernel of degree 3
poly_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel='poly', degree=3, C=1.0)
)
poly_svm.fit(X_train, y_train)
print(f"Poly SVM accuracy: {poly_svm.score(X_test, y_test):.3f}")
```

Support Vector Regression (SVR)

SVM concepts extend naturally from classification to regression. In **Support Vector Regression (SVR)**, the margin concept is replaced by an **ε-insensitive tube** surrounding the regression function $f(\mathbf{x})$.

ε-Insensitive Loss: Predictions falling *inside* the tube (within ε of the true target) incur **zero loss**. Only predictions *outside* the tube pay a linear penalty via slack variables $\xi_i, \hat{\xi}_i \geq 0$ (one for each direction—above and below the tube).

The SVR primal optimization problem is:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}, \hat{\boldsymbol{\xi}}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}(\xi_i + \hat{\xi}_i)$$

$$\text{s.t.} \quad y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)} - b \leq \varepsilon + \xi_i, \quad \mathbf{w}^T\mathbf{x}^{(i)} + b - y^{(i)} \leq \varepsilon + \hat{\xi}_i, \quad \xi_i, \hat{\xi}_i \geq 0$$

Like classification SVMs, SVR relies on support vectors (the points on or outside the tube boundary) and can be fully **kernelized** to fit non-linear regression curves.

```python
# Support Vector Regression (SVR)
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

svr_model = make_pipeline(
    StandardScaler(),
    SVR(kernel='rbf', C=100, epsilon=0.1)  # eps-insensitive tube
)
svr_model.fit(X_train, y_train)
y_pred = svr_model.predict(X_test)

from sklearn.metrics import mean_squared_error
print(f"SVR MSE: {mean_squared_error(y_test, y_pred):.4f}")
```

Logistic Regression

A Probabilistic View of Classification

Previous chapters framed classification geometrically: find a hyperplane that separates classes. Logistic Regression introduces a **probabilistic formulation**—instead of predicting a hard label $y \in \{0, 1\}$, we model the **conditional probability** that the label is y given input $\mathbf{x}$:

$$P(y \mid \mathbf{x})$$

This is strictly more informative: it yields the most likely label *and* quantifies the model's uncertainty in that prediction.

Generative Models: Learn the joint $P(\mathbf{x}, y)$ or the class-conditional $P(\mathbf{x} \mid y)$, then apply Bayes' rule to obtain $P(y \mid \mathbf{x}) = \frac{P(\mathbf{x}|y)\,P(y)}{P(\mathbf{x})}$. They model *how the data was generated* (e.g., Naive Bayes).

Discriminative Models: Learn $P(y \mid \mathbf{x})$ directly, bypassing the full data distribution. **Logistic Regression is a discriminative model.**

The Logistic Sigmoid Function

A linear model computes a score $z = \mathbf{w}^T\mathbf{x} + w_0 \in (-\infty, +\infty)$. To interpret this as a probability, we need a mapping to $[0, 1]$.

Definition 57 (Logistic Sigmoid Function)**.** The sigmoid function $\sigma(z)$ squashes any real number into the interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

Key Properties of $\sigma(z)$:

- $\sigma(z) \to 1$ as $z \to +\infty$; $\quad \sigma(z) \to 0$ as $z \to -\infty$; $\quad \sigma(0) = 0.5$.
- **Symmetry:** $\sigma(-z) = 1 - \sigma(z)$.
- Nearly linear around $z = 0$, so small changes in the score produce proportional changes in probability.
- Flattens toward the extremes, squashing outlier scores toward 0 or 1.
- **Differentiable**, with a clean derivative: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$.

```
import numpy as np
import matplotlib.pyplot as plt

# Plot the sigmoid function
z = np.linspace(-7, 7, 300)
sigma = 1 / (1 + np.exp(-z))          # sigmoid formula

plt.plot(z, sigma, linewidth=2)
plt.axhline(0.5, color='gray', linestyle='--', linewidth=0.8)
plt.axvline(0.0, color='gray', linestyle='--', linewidth=0.8)
plt.xlabel("z (logit)")
plt.ylabel(r"$\sigma(z)$")
plt.title("Logistic Sigmoid Function")
plt.show()
```

Remark 66 (Logit: The Inverse of the Sigmoid)**.** The input z to the sigmoid is called the **logit**. The logit function is the *inverse* of the sigmoid and equals the log of the odds ratio:

$$\text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1 - p}$$

Interpreting z as a log-odds gives probabilistic meaning to the linear score: a score of $z = 2$ means the log-odds of class 1 are 2, i.e., $P(y{=}1)/P(y{=}0) = e^2 \approx 7.4$.

The Logistic Regression Model

For binary classification ($y \in \{0, 1\}$), Logistic Regression models the probability of the positive class as:

$$P(y{=}1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + w_0) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+w_0)}}$$

and consequently:

$$P(y{=}0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w}^T\mathbf{x} + w_0) = \sigma(-(\mathbf{w}^T\mathbf{x} + w_0))$$

Both cases can be written compactly using the Bernoulli form:

$$P(y \mid \mathbf{x}, \mathbf{w}, w_0) = \hat{y}^{\,y}\,(1 - \hat{y})^{1-y}, \quad \hat{y} = \sigma(\mathbf{w}^T\mathbf{x} + w_0)$$

Remark 67 (The Decision Boundary Is Linear)**.** To make a hard prediction, we apply a threshold at 0.5: predict $y{=}1$ if $P(y{=}1 \mid \mathbf{x}) \geq 0.5$. Since $\sigma(z) \geq 0.5 \iff z \geq 0$, the rule simplifies to:

$$\text{Predict 1 if } \mathbf{w}^T\mathbf{x} + w_0 \geq 0$$

Despite the non-linear sigmoid, **the decision boundary is the hyperplane $\mathbf{w}^T\mathbf{x} + w_0 = 0$**—exactly where both classes are equiprobable.

```python
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate a binary classification dataset
X, y = make_classification(n_samples=300, n_features=2,
                           n_redundant=0, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Fit logistic regression
model = LogisticRegression()
model.fit(X_train, y_train)

print(f"Weights w:  {model.coef_[0]}")
print(f"Bias w_0:   {model.intercept_[0]:.4f}")
print(f"Accuracy:   {model.score(X_test, y_test):.3f}")
```

```python
# predict_proba returns [P(y=0|x), P(y=1|x)] for each sample
probs = model.predict_proba(X_test[:5])
for i, (p0, p1) in enumerate(probs):
    label = model.predict(X_test[i:i+1])[0]
    print(f"Sample {i}: P(0)={p0:.3f}  P(1)={p1:.3f} -> y_hat={label}")
```

Training: Maximum Conditional Likelihood

Because Logistic Regression outputs probabilities, we train it by **Maximum Conditional Likelihood Estimation (MCLE)**: find $\mathbf{w}, w_0$ that make the observed labels most probable under the model.

Given N i.i.d. training pairs, the conditional likelihood is:

$$L(\mathbf{w}, w_0) = \prod_{i=1}^{N} P(y^{(i)} \mid \mathbf{x}^{(i)}, \mathbf{w}, w_0) = \prod_{i=1}^{N} \hat{y}^{(i)\,y^{(i)}}\,(1 - \hat{y}^{(i)})^{1-y^{(i)}}$$

Taking the negative log converts the product to a sum and flips to minimization.

Definition 58 (Binary Cross-Entropy / Log Loss)**.** The Logistic Regression cost function is the negative log-likelihood, averaged over N samples:

$$\mathcal{L}(\mathbf{w}, w_0) = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)}\log\hat{y}^{(i)} + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})\right]$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T\mathbf{x}^{(i)} + w_0)$.

Why this loss works: When $y = 1$, only the $-\log \hat{y}$ term is active—if the model confidently predicts $\hat{y} \approx 0$, the loss $-\log(\hat{y}) \to \infty$, heavily penalizing the wrong confident prediction. Symmetrically for $y = 0$.

Remark 68 (Equivalence to ERM)**.** Minimizing the negative log-likelihood is equivalent to Empirical Risk Minimization (as established in the ERM framework) with the log-loss as the per-sample loss function. The probabilistic and optimization views of ML converge here.

Optimization

Setting $\nabla\mathcal{L} = 0$ yields no closed-form solution due to the non-linear e^{-z} term inside σ. We must use **iterative optimization**.

Key fact: The log-loss is a **strictly convex** function of $\mathbf{w}$ (it is a linear combination of convex functions). This guarantees:

- Every local minimum is the global minimum.
- Gradient descent converges to the unique optimal $\mathbf{w}^*$ (given appropriate learning rate).

The gradient of the log-likelihood (for gradient *ascent*) has a remarkably clean form:

$$\nabla_{\mathbf{w}}\, \ell(\mathbf{w}) = \sum_{i=1}^{N} \left(y^{(i)} - \sigma(\mathbf{w}^T\mathbf{x}^{(i)} + w_0)\right) \mathbf{x}^{(i)}$$

Each term is the prediction error $(y - \hat{y})$ scaled by the input $\mathbf{x}$—the same structure as in linear regression.

Regularization

Remark 69 (Why Regularize?)**.** If training data is **perfectly linearly separable**, the model can drive the loss to zero by pushing $\hat{y} \to 1$ or $\hat{y} \to 0$ for every sample. This requires $\|\mathbf{w}\| \to \infty$, producing an infinitely steep sigmoid that memorizes the training data but generalizes poorly.

Adding a penalty term prevents weight explosion:

L_2 **(Ridge):** $\mathcal{L}_{\text{reg}} = \mathcal{L}(\mathbf{w}) + \beta \sum_j w_j^2$ — prefers many small weights; smooth optimization.

L_1 **(Lasso):** $\mathcal{L}_{\text{reg}} = \mathcal{L}(\mathbf{w}) + \alpha \sum_j |w_j|$ — drives some weights to exactly zero (feature selection).

In `sklearn`, regularization strength is controlled by the parameter `C`, where `C` $= 1/\lambda$. Smaller `C` means stronger regularization.

```
from sklearn.linear_model import LogisticRegression
import numpy as np

# Compare regularization strengths via the C parameter
# C = 1/lambda: smaller C = stronger regularization
for C_val in [0.01, 1.0, 100.0]:
    model = LogisticRegression(C=C_val, penalty='l2', max_iter=1000)
    model.fit(X_train, y_train)
    w_norm = np.linalg.norm(model.coef_)
    acc = model.score(X_test, y_test)
    print(f"C={C_val:6.2f}  ||w||={w_norm:.4f}  Acc={acc:.3f}")
# Strong regularization (small C) -> smaller weights, may underfit
# Weak regularization (large C) -> larger weights, may overfit
```

Multi-Class Extension: Softmax Regression

Binary Logistic Regression generalizes to K classes via the **softmax function**. Instead of a single weight vector, we learn a weight matrix $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_K]$ with one vector per class:

$$P(y{=}k \mid \mathbf{x}, \mathbf{W}) = \text{softmax}(\mathbf{W}, \mathbf{x}, k) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}}$$

The softmax normalizes the K raw scores into a valid probability distribution that sums to 1. For $K{=}2$, this reduces exactly to the sigmoid.

The corresponding loss is the **categorical cross-entropy**:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbf{1}[y^{(i)}{=}k] \ \log P(y{=}k \mid \mathbf{x}^{(i)})$$

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Iris: 3 classes -> softmax is used automatically
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=0)

# multi_class='multinomial' uses softmax (default in sklearn)
model = LogisticRegression(multi_class='multinomial', max_iter=200)
model.fit(X_tr, y_tr)

print(f"Classes:  {model.classes_}")
print(f"Accuracy: {model.score(X_te, y_te):.3f}")
# Each row of predict_proba sums to 1 across all K classes
print(f"Probs (first sample): {model.predict_proba(X_te[:1])[0]}")
```

Decision Trees

What Are Decision Trees?

A **decision tree** is a hierarchical, tree-structured model for both classification and regression. It learns a set of *if-then-else* rules directly from data, making it one of the most interpretable models in machine learning.

Internal nodes: Each tests a single feature against a threshold (e.g., "horsepower ≤ 130?").

Branches: Represent the outcomes of the test (yes/no for binary splits).

Leaves: Contain the final prediction—a class label (classification) or a numeric value (regression).

Remark 70 (Key Properties)**.** Decision trees are **non-parametric**: they make no assumptions about the underlying data distribution. They can capture **non-linear decision boundaries**, but only through **axis-aligned** (perpendicular to feature axes) splits. No feature scaling is required, and they naturally handle both continuous and categorical features.

To classify or predict for a new input $\mathbf{x}$, we simply traverse the tree from root to leaf, following the branch corresponding to each feature test. The leaf we arrive at provides the prediction.

Building a Decision Tree — Recursive Partitioning

Decision trees are built using a **greedy, top-down, recursive** strategy (divide-and-conquer). The classic algorithm is ID3 (Quinlan, 1986), later refined into C4.5 and CART:

1. **Start** with the entire training dataset at the root node.
2. **Select the best feature and threshold** to split on—the one that produces the "purest" child partitions.
3. **Partition** the data into child nodes based on the split.
4. **Recurse** on each child node that is not yet pure.
5. **Stop** when a stopping criterion is met: all samples in a node share the same label (pure node), a maximum depth is reached, or the number of samples falls below a threshold.

The greedy nature means the algorithm makes the locally optimal split at each node without backtracking. This does *not* guarantee a globally optimal tree, but is computationally efficient and works well in practice.

Splitting Criteria for Classification

The central question at each node: *which feature should we split on?* We want splits that maximally reduce the **impurity** of child nodes.

Entropy and Information Gain

Definition 59 (Entropy)**.** For a node containing samples with label distribution $p_1, p_2, \ldots, p_K$ over K classes, the **entropy** is:

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k$$

where p_k is the fraction of samples belonging to class k. By convention, $0 \log_2 0 = 0$.

Entropy measures uncertainty: $H = 0$ for a **pure node** (all samples one class), and H is maximized when all classes are equally represented (e.g., $H = 1$ bit for a 50/50 binary split).

Definition 60 (Information Gain)**.** The **information gain** from splitting a set S on attribute A is:

$$\text{Gain}(S, A) = H_S(Y) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \cdot H_{S_v}(Y)$$

where S_v is the subset of S where attribute A takes value v. The second term is the weighted average entropy of the children. ID3 selects the attribute A with the **highest information gain**—the greatest reduction in uncertainty.

Gini Impurity

Definition 61 (Gini Impurity)**.** An alternative measure of node impurity used by the CART algorithm:

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

This equals the probability that a randomly chosen sample would be **misclassified** if labeled according to the class distribution of the node.

For binary classification: $G = 0$ when pure, $G = 0.5$ at maximum impurity (50/50 split). Gini is slightly faster to compute than entropy (no logarithm), and in practice the two criteria produce nearly identical trees.

Choosing the Best Split (Continuous Features)

For a continuous feature, we cannot enumerate discrete values. Instead, the algorithm sorts all training values for that feature, considers every midpoint between consecutive distinct values as a candidate threshold t, and evaluates the binary split "feature $\leq t$" vs. "feature $> t$". The threshold yielding the highest information gain (or largest Gini reduction) is selected.

Splitting Criteria for Regression

For **regression trees**, labels are continuous, so entropy-based measures are meaningless. Instead, we use **variance reduction** as the measure of partition quality.

Definition 62 (Variance Reduction)**.** At each node, we measure impurity using the **variance** (or MSE) of the target values. The best split minimizes the weighted sum of child variances:

$$\text{Gain}(S, A, t) = \text{Var}(S) - \left(\frac{|S_L|}{|S|} \text{Var}(S_L) + \frac{|S_R|}{|S|} \text{Var}(S_R) \right)$$

where S_L, S_R are the left and right partitions after splitting on feature A at threshold t, and:

$$\text{Var}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

The prediction at each leaf is the **mean** of the target values of training samples in that region. Because partitions have low variance, the mean is a good representative of the set.

Remark 71. The deeper the regression tree, the more partitions it creates. With N training points and no depth limit, the tree can create up to N partitions—each containing a single point—perfectly memorizing the training data (overfitting).

Overfitting and Pruning

An unconstrained decision tree will keep splitting until every leaf is pure (classification) or contains a single sample (regression). This produces a tree that memorizes the training data but generalizes poorly—classic **overfitting**. Even irrelevant features will be used for splitting when noise is present.

Pre-Pruning (Early Stopping)

Constrain tree growth *during* construction via hyperparameters:

`max_depth`: Maximum depth of the tree. Lower values yield simpler, more regularized models.

`min_samples_split`: Minimum number of samples required to split an internal node.

`min_samples_leaf`: Minimum number of samples required in each leaf node.

`max_leaf_nodes`: Maximum number of leaf nodes in the tree.

Post-Pruning (Cost-Complexity Pruning)

Grow the full tree first, then prune branches that do not improve validation performance. **Cost-complexity pruning** (also called minimal cost-complexity pruning) minimizes:

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

where $R(T)$ is the training error (e.g., misclassification rate or MSE), $|T|$ is the number of leaf nodes, and $\alpha \geq 0$ is a complexity parameter. Larger α penalizes complex trees more aggressively, producing smaller trees. In scikit-learn, this is controlled by the `ccp_alpha` hyperparameter.

Decision Trees in Practice with scikit-learn

Classification

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=99)

# Train with entropy criterion, max depth = 3
dt = DecisionTreeClassifier(
    criterion='entropy', max_depth=3, random_state=99)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")

# Feature importances (based on total impurity reduction)
for name, imp in zip(feature_names, dt.feature_importances_):
    print(f"  {name}: {imp:.3f}")
```

Controlling Overfitting

```
# Pre-pruning: constrain depth and leaf size
dt_shallow = DecisionTreeClassifier(
    max_depth=4, min_samples_leaf=5, random_state=99)
dt_shallow.fit(X_train, y_train)
print(f"Depth-4 test acc: "
      f"{accuracy_score(y_test, dt_shallow.predict(X_test)):.3f}")

# Post-pruning: cost-complexity pruning with ccp_alpha
dt_pruned = DecisionTreeClassifier(
    ccp_alpha=0.02, random_state=99)   # larger alpha = simpler tree
dt_pruned.fit(X_train, y_train)
print(f"Pruned test acc:  "
      f"{accuracy_score(y_test, dt_pruned.predict(X_test)):.3f}")
```

Regression

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

dt_reg = DecisionTreeRegressor(max_depth=4, random_state=99)
dt_reg.fit(X_train, y_train)

y_pred = dt_reg.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
print(f"RMSE: {rmse:.3f}")      # leaf predictions = mean of region
print(f"R^2:  {dt_reg.score(X_test, y_test):.3f}")
```

Pros and Cons

Pros:

- Highly **interpretable**—the tree can be read as a set of if-then rules.
- Handles both **continuous and categorical** features natively.
- Requires **no feature scaling** (splits are threshold-based).
- Captures **non-linear** relationships and feature interactions.
- Fast training and prediction.

Cons:

- **High variance**: small changes in training data can produce a completely different tree.
- **Greedy** construction does not guarantee the globally optimal tree.
- Decision boundaries are **axis-aligned only**—cannot represent diagonal boundaries efficiently.
- **Prone to overfitting** without careful pruning or depth control.
- Can be biased toward features with many distinct values.

From Trees to Forests: Ensemble Methods

The high variance of individual decision trees is their Achilles' heel. **Ensemble methods** address this by combining many trees into a single, more robust predictor.

Bagging and Random Forests

Bagging (Bootstrap Aggregating) trains B independent trees, each on a different bootstrap sample (random sample with replacement) of the training data, and averages their predictions. **Random Forests** extend bagging by additionally randomizing the feature subset considered at each split, further decorrelating the trees.

The result: dramatically **reduced variance** with little increase in bias, at the cost of interpretability.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,       # number of trees
    max_depth=5,            # depth of each tree
    random_state=99)
rf.fit(X_train, y_train)

print(f"RF test acc: "
      f"{accuracy_score(y_test, rf.predict(X_test)):.3f}")
```

Boosting

Boosting builds trees *sequentially*, where each new tree focuses on correcting the mistakes of the ensemble so far. Methods include **AdaBoost** (re-weights misclassified samples) and **Gradient Boosting** (fits each new tree to the residual errors). Boosting reduces *both* bias and variance and is widely considered the **state-of-the-art for tabular data**.

Remark 72 (Practical Guideline)**.** A single decision tree is rarely competitive with other models. Its real power lies in serving as the building block for ensemble methods—Random Forests and Gradient Boosting (XGBoost, LightGBM)—which dominate competitions and real-world applications on structured/tabular data.

Neural Networks 1: Multi-Layer Perceptron (MLP)

From Biological Neurons to Artificial Units

Artificial Neural Networks (ANNs) are inspired by the idea of mimicking the human brain through a parallel and distributed network of activation units[3].

Definition 63 (Artificial Neuron)**.** The fundamental building block of an ANN is the artificial neuron, which mimics biological neurons and synaptic connections[5]. It consists of three main components:

1. **Input function (linear):** Computes a weighted sum of the inputs[5, 6].
2. **Activation function (non-linear):** Applied to the linear input[6].
3. **Output:** The resulting value of the activation is passed on via fan-out[6].

The earliest model of a linear classifier, the **Perceptron**, was designed by F. Rosenblatt in the 1950s-60s[6]. A neuron activates and fires (e.g., passing output from 0 to 1) if the sum of its inputs reaches a certain internal threshold[7].

Neural Networks as Function Approximators

At their core, neural networks are parametric non-linear function approximators[14].

Definition 64 (Network Goal)**.** Given a dataset of training examples, a neural network aims to learn a generic non-linear mapping $f : X \to Y$[4]. It estimates the target function through a parametric estimator $f_{\mathbf{w}}$, parameterized by its connection weights $\mathbf{w}$[4, 5]. The goal of training is to iteratively change the weights $\mathbf{w}$ to minimize a loss function, such as Mean Squared Error (MSE), over the training dataset[15, 16]. This is typically achieved using gradient descent[16].

Network Topologies

Feed-Forward (FF) Networks: Data processing proceeds sequentially through stages—from the input layer, through one or more intermediate **hidden layers**, to the output layer[16]. Examples include Multi-Layer Perceptrons (MLPs) which use fully connected dense layers, and Convolutional Neural Networks (CNNs)[16, 17].

Recurrent Neural Networks (RNNs): Contain self-recurrent inputs that allow the network to maintain a state[17, 18]. When unrolled in time, they are highly effective for learning from sequences, time series, and text or video classification[18].

The Multi-Layer Perceptron (MLP) and Feature Learning

An MLP is a deep feed-forward network consisting of fully connected layers, meaning each unit is fully connected to the units of the preceding layer[16, 17].

Remark 73 (Why Non-Linear Activations?)**.** Hidden layers allow the network to learn non-linear functions[16]. A multi-layer network constructed entirely of linear units would lack this capacity. To learn complex patterns, **non-linear activation functions** are essential. While the sigmoid function (akin to logistic regression) is a classic choice, it is not the only option available for modern networks[22, 24].

Hidden Layers as Feature Extractors

The true power of an MLP lies in its hidden layers, making neural networks excellent feature learners[24, 26].

- Each hidden unit takes feature values as input and performs a non-linear feature extraction[24].
- The collective outputs from all units in a hidden layer generate a **feature vector**, which acts as the input for the subsequent layer[24].
- Through multiple stages, the network recursively generates features, learning a feature map $\boldsymbol{\phi}(\mathbf{x})$ that represents a highly non-linear transformation of the original inputs[24].

Mathematical Complexity and Design Choices

An MLP represents a highly non-linear, **non-convex** composite function[27, 28].

Designing an effective MLP requires tuning several crucial architectural elements[28]:

1. **Representational Power:** The number of hidden units determines the network's capacity to represent complex patterns[28].
2. **Architecture Shape:** Deciding whether the network should be *wide* (many units) or *deep* (many hidden layers)[28].
3. **Activation Functions:** Selecting the appropriate non-linear function for the units[28].

Neural Networks 2: MLPs, Design and Optimization

The Optimization Problem

Training a Neural Network is fundamentally an optimization problem. The goal is to find a set of weights $\mathbf{w}$ such that the network's parameterized function $f_{\mathbf{w}}(X)$ closely approximates the true function $f(X)$ over a training dataset $\mathcal{D}$.

Definition 65 (Empirical Risk Minimization)**.** We iteratively update the weights $\mathbf{w}$ to minimize a specified loss function $L(f_{\mathbf{w}}(\mathbf{x}), y)$. For regression, a common choice is the **Expected Squared Error** over the dataset:

$$E_{\mathcal{D}}[\mathbf{w}] = \frac{1}{2}\sum_{i=1}^{m}(f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Because an MLP is a composite non-linear function, this error surface is highly **non-convex**, containing many local minima, plateaus, and steep valleys[cite: 455, 456].

Gradient Descent and Backpropagation

Since analytical solutions for the global minimum are impossible for such non-convex surfaces, we use **Gradient Descent (GD)**. We calculate the gradient of the error function $\nabla E(\mathbf{w})$ and take a step in the opposite direction:

$$\Delta\mathbf{w} = -\eta\nabla E(\mathbf{w})$$

where η is the learning rate (or step size)[cite: 323].

Backpropagation (The Chain Rule)

Because the network is a deep composite function (e.g., $h(\mathbf{x}) = g(f(\mathbf{x})))$, we cannot immediately compute the gradient of the error with respect to the weights in the earliest layers[cite: 339]. We use **Backpropagation**, which is an efficient application of the **chain rule** of calculus.

- We first compute a forward pass to get the network's prediction and the loss.
- We then propagate the error backwards, computing local derivatives at each layer.
- The chain rule multiplies these local gradients recursively (e.g., $\frac{\partial h}{\partial x} = \frac{\partial h}{\partial u}\frac{\partial u}{\partial x}$), yielding the gradient for every single weight in the network[cite: 336, 339].

Gradient Descent Implementations

How much data we use to compute $\nabla E(\mathbf{w})$ before taking a step defines the GD variant[cite: 324, 325]:

Batch Gradient Descent: Computes the gradient over the *entire* dataset. Slow per step and computationally heavy.

Stochastic Gradient Descent (SGD): Computes the gradient and updates weights using a *single* data point. Fast but yields a highly noisy trajectory.

Mini-batch SGD: Computes the gradient over a subset of size $N \ll m$. This is the standard in deep learning, as it balances computational efficiency (via vectorization) and gradient stability.

Advanced Optimizers

Standard SGD can struggle with pathological curvatures (like steep valleys) or noisy gradients. Modern training relies on advanced optimizers:

- **Momentum:** Accumulates past gradients (velocity) to smooth out the optimization trajectory. It accelerates convergence in consistent directions and dampens oscillations[cite: 356, 359].

- **Nesterov Accelerated Gradient (NAG):** Looks ahead to where the momentum would carry the weights, and calculates the gradient at *that* future point. This helps prevent overshooting the optimal solution[cite: 368].
- **Adam (Adaptive Moment Estimation):** The standard for training modern networks. It combines momentum (1st statistical moment of the gradients) with an adaptive learning rate (2nd statistical moment of squared gradients). It ensures stable updates and corrects initial biases[cite: 369, 370].

Multi-Class Classification: Softmax and Cross-Entropy

For multi-class classification (e.g., MNIST digit recognition), the network's final layer outputs a vector of raw scores (logits) $\mathbf{z}$[cite: 377, 385].

Definition 66 (Softmax Function)**.** The Softmax function squashes the K-dimensional vector of real-valued logits into a normalized probability distribution (values in $[0, 1]$ that sum to 1)[cite: 377, 378].

Definition 67 (Cross-Entropy Loss)**.** Used in tandem with Softmax. It measures the divergence between the true one-hot encoded label distribution $\mathbf{y}$ and the predicted probabilities $\hat{\mathbf{y}}$[cite: 383, 385]. For a single sample with true class c:

$$\text{CEL} = -\log(\hat{y}_c)$$

This cleanly penalizes confident incorrect predictions. The exponential in Softmax pairs perfectly with the logarithm in CEL, simplifying gradient calculations and avoiding numerical overflow[cite: 383, 384].

Training Dynamics and MLP Limitations

Remark 74 (Epochs)**.** Learning is organized in **Epochs**. One epoch represents one full sweep of the entire training dataset using the mini-batch updates[cite: 395]. Balancing the number of epochs and the batch size is critical to prevent overfitting (too many epochs) or underfitting (too few)[cite: 395].

Remark 75 (Limitations of Fully-Connected MLPs)**.** When dealing with structured data like images, MLPs show significant weaknesses[cite: 408, 409]:

1. **Insensitivity to Input Order:** An image must be flattened into a 1D vector. The MLP ignores spatial correlations; shuffling all pixels identically across the dataset yields the exact same performance[cite: 410].
2. **High Parameter Count:** Dense connections lead to a massive number of weights (e.g., millions for small images), increasing memory constraints and the risk of overfitting[cite: 408].

These limitations motivate architectures with structural awareness, such as Convolutional Neural Networks (CNNs).

Neural Networks 3: Convolutional Neural Networks (CNN)

Limitations of MLPs for Structured Data

Traditional Multi-Layer Perceptrons (MLPs) take a single 1D column vector as input. When dealing with structured, high-dimensional data like images, this architecture presents two major issues[cite: 4, 6]:

1. **Space-Time Complexity:** An MLP requires fully connected layers. For a 1000×1000 RGB image, flattening yields 3×10^6 inputs. Connecting this to a hidden layer of 1 million units requires 3×10^{12} parameters (weights)[cite: 3]. This leads to unmanageable memory constraints and high risks of overfitting.
2. **Destruction of Spatial Structure:** By flattening an image into a 1D array, the MLP ignores local spatial correlations (e.g., adjacent pixels forming an edge)[cite: 5, 8]. If you consistently shuffle all the pixels in an image dataset, an MLP's final performance remains exactly the same, highlighting its insensitivity to structural order[cite: 7].

Core Principles of CNNs

Convolutional Neural Networks (CNNs) are explicitly designed to leverage the inherent spatial and temporal coherence of data (like images, audio, and text)[cite: 12].

Definition 68 (Local Connectivity & Receptive Fields)**.** Instead of connecting every input pixel to a hidden unit, a CNN unit is only connected to a small, localized region of the input space known as its **receptive field** (e.g., a 3×3 or 5×5 patch)[cite: 20]. This dramatically reduces the number of operations and parameters[cite: 13].

Definition 69 (Weight Sharing & Feature Maps)**.** A CNN applies a **convolutional filter** (or kernel) across the entire image[cite: 20, 24]. The weights and bias of this filter are **shared** for all spatial locations[cite: 22]. This assumes that if a feature (like a vertical edge) is useful to detect in one part of the image, it is equally useful in another[cite: 27, 28, 29]. The output of applying one filter across the image is called a **feature map**[cite: 24, 25].

Key Architectural Layers

A standard CNN pipeline alternates feature extraction layers with dimensionality reduction, concluding with a standard classification head[cite: 10, 38].

1. Convolutional Layer

This layer applies multiple filters (a *filter bank*) to the input[cite: 31, 32]. Each filter extracts a different type of feature, resulting in multiple, parallel feature maps (often representing a 3D volume of features, or channels)[cite: 24, 31].

- **Stride:** The step size by which the filter slides across the image[cite: 20].
- **Padding:** Adding zeros around the borders of the input to control the spatial dimensions of the output feature map[cite: 21].

2. Pooling Layer

Usually applied immediately after a convolutional layer, pooling acts as a subsampling mechanism to compress the feature maps[cite: 33, 34].

- **Max-Pooling:** Outputs the maximum activation within a given region (e.g., a 2×2 window)[cite: 35].
- **Purpose:** It provides translational invariance—once a feature is detected, its exact location is less important than its relative presence. It also significantly reduces the spatial dimensions and parameter count for subsequent layers[cite: 36, 37].

3. Flattening and Dense (Fully-Connected) Layers

After hierarchical feature extraction, the 3D volume of feature maps is **flattened** into a 1D vector[cite: 38]. This vector is fed into a standard dense MLP, which acts as the final probabilistic classifier (often ending in a Softmax activation)[cite: 38, 40].

Remark 76 (Joint Training)**.** The beauty of a CNN is that the feature extractor (convolutional + pooling layers) and the classifier (dense layers) are **jointly trained** end-to-end via backpropagation[cite: 40]. The network learns the optimal filters directly from the data.

Parameter Efficiency Example

To understand the efficiency of weight sharing, consider an input image of 28×28 pixels.

- A convolutional layer applying 32 distinct 3×3 filters requires only:
  $(3 \times 3 \text{ weights} + 1 \text{ bias}) \times 32 \text{ filters} =$ **320 parameters**[cite: 60, 61].
- A fully-connected dense layer mapping the resulting 4608 flattened features to 128 hidden units requires:
  $(4608 \times 128) + 128 \text{ biases} =$ **589,952 parameters**[cite: 64].

This demonstrates how convolutional layers extract rich representations for a tiny fraction of the parameter cost of dense layers.

Transfer Learning

Because training deep CNNs from scratch requires vast amounts of data and compute, **Transfer Learning** is standard practice. A network pre-trained on a massive dataset (like ImageNet) can be reused for new tasks:

Fixed Feature Extractor: The pre-trained convolutional layers are frozen. Only the final dense classification head is replaced and trained on the new, smaller dataset.

Fine-Tuning: The final dense layers are replaced, and the error gradients are backpropagated through the entire network, subtly adjusting (fine-tuning) the pre-trained convolutional weights for the new domain.

Natural Language Processing, Transformers, and GenAI

Introduction to Natural Language Processing (NLP)

Definition 70 (NLP)**.** Natural Language Processing (NLP) is an interdisciplinary subfield of Linguistics, Computer Science, and Artificial Intelligence. It focuses on the interactions between computers and human language, aiming to program computers to process, analyze, and generate natural language data.

NLP generally encompasses two main sub-goals:

- **Natural Language Understanding (NLU):** Enabling computers to comprehend text and extract semantic meaning (e.g., Information Extraction, Sentiment Analysis, Text Classification).
- **Natural Language Generation (NLG):** Enabling computers to create human-like language outputs (e.g., Translation, Summarization, Dialogue generation).

Evolution of NLP Techniques

The field of NLP has evolved through several distinct paradigms:

1. **Rule-based Systems:** Relied on regular expressions and hand-crafted grammar rules (e.g., subject-verb agreement). *Limitations:* Highly brittle, difficult to scale, and cannot generalize beyond explicitly defined rules.
2. **Statistical NLP:** Utilized probability distributions, n-grams (probability of the next word given n previous words), and metrics like TF-IDF (Term Frequency-Inverse Document Frequency). *Limitations:* Lacks a deep understanding of semantic meaning beyond raw word frequencies.
3. **Neural NLP (RNNs/LSTMs):** Transitioned to representing words as dense vectors (embeddings) and processing them sequentially. *Limitations:* Sequential processing is slow, struggles with long-term memory loss (forgetting distant context), and forces all information through a single hidden state bottleneck.
4. **Transformers:** Replaced sequential processing with parallel attention mechanisms, allowing the model to learn relationships between any pair of words instantly. They are highly scalable and serve as the foundation for modern Large Language Models (LLMs).

Tokenization and Embeddings

Before text can be ingested by a neural network, it must be converted into numerical representations through a two-step process.

1. Tokenization

Tokenization breaks raw text into smaller, processable units known as tokens.

- **Character-level:** Features a very small vocabulary and eliminates the Out-Of-Vocabulary (OOV) problem. However, individual characters lack semantic meaning and result in excessively long sequences.
- **Word-level:** Tokens carry clear semantic meaning, but the vocabulary size explodes. It handles rare or misspelled words poorly (the OOV problem) and ignores word structures (prefixes/suffixes).
- **Sub-word (e.g., WordPiece):** The modern standard. It balances vocabulary size by generating sub-word units based on frequency. Frequent words remain intact, while rare words are broken into meaningful sub-components (e.g., "playing" $\rightarrow$ "play" + "ing"), effectively handling unseen words while maintaining semantic density.

2. Word Embeddings

Embeddings map discrete tokens to continuous numeric vectors.

- **Bag of Words (BoW) / One-Hot Encoding:** Creates sparse, high-dimensional vectors that ignore word order and fail to capture semantic similarity.

- **Word2Vec (Neural Embeddings):** Learns low-dimensional, dense vector representations based on context prediction. It successfully captures semantic meaning, ensuring that words with similar contexts are mapped closely together in the vector space.

The Transformer Architecture

Definition 71 (Self-Attention)**.** The core innovation of the Transformer. Instead of processing words sequentially, every word in a sequence attends to all other words simultaneously to gather context. Learned **attention scores** determine how much each word contributes to another's meaning, producing a rich, context-aware embedding. **Multi-head attention** runs several of these mechanisms in parallel to capture diverse linguistic patterns simultaneously.

The Transformer Block

A standard Transformer block processes features through the following stages:

1. **Positional Encoding:** Added to input embeddings to retain the structural order of words, which would otherwise be lost during parallel processing.
2. **Self-Attention:** Captures contextual relationships across the sequence.
3. **Add & Normalize:** A residual connection combined with layer normalization to stabilize and accelerate training.
4. **Feed Forward Network:** Applies non-linear transformations to the features.
5. **Add & Normalize:** A final stabilization step before passing the output to the next block.

Transformer Topologies

Encoder-only (e.g., BERT): Processes the full input sequence simultaneously (bidirectional). Excellent for NLU tasks where complete context is required, such as sentiment classification or semantic search.

Decoder-only (e.g., GPT): Future words are masked to strictly preserve generation order. The model predicts the sequence auto-regressively (step-by-step). This is the standard architecture for modern generative LLMs.

Encoder-Decoder (e.g., T5): The Encoder builds a full contextual representation of the input, while the Decoder generates the output step-by-step using cross-attention to reference the Encoder's state. Ideal for sequence-to-sequence tasks like translation and summarization.

Generative AI: Autoencoders and GANs

Generative AI principles extend beyond text to the synthesis of complex continuous data, such as images.

Definition 72 (Autoencoders)**.** Neural networks designed to learn efficient, compressed representations (embeddings) of data in an unsupervised manner.

- **Encoder:** Compresses the high-dimensional input data (e.g., an image) into a low-dimensional *latent vector*.
- **Decoder:** Attempts to rebuild the original data from the compressed latent vector.
- **Training Goal:** Minimize the **Reconstruction Loss** (the mathematical difference between the original input and the reconstructed output).

Definition 73 (Generative Adversarial Networks (GANs))**.** A generative framework where two neural networks are trained simultaneously in a zero-sum adversarial game.

- **Generator (G):** Takes a random noise vector as input and attempts to synthesize a realistic data sample (e.g., a forged handwritten digit).
- **Discriminator (D):** Takes an input image (which has a 50% chance of being a real dataset image and a 50% chance of being generated by G) and predicts whether it is real or fake.

Through continuous competition, the Generator learns to map random noise into highly realistic, structured data distributions to fool the Discriminator.

Practical Applications: Transformers, LLMs, and GANs

The Hugging Face Ecosystem

While understanding the underlying mathematical and architectural principles of Transformers is crucial, modern Machine Learning workflows heavily rely on high-level libraries for efficiency. **Hugging Face** has emerged as the standard ecosystem for Natural Language Processing and Generative AI.

Definition 74 (Hugging Face Libraries)**.** The ecosystem is primarily built around two core libraries:

- **`transformers`:** Provides thousands of pre-trained models (like BERT, GPT, T5) to perform tasks on texts, images, and audio. It abstracts away the complex PyTorch/TensorFlow implementations into simple APIs.
- **`datasets`:** A highly efficient library designed to easily download, process, and share massive datasets used for training and evaluating machine learning models.

Working with BERT-Based Models

BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only architecture excelling at Natural Language Understanding (NLU) tasks.

Tokenization in Practice

Before feeding raw text to a BERT model, it must be tokenized and formatted exactly as the model expects.

```
from transformers import AutoTokenizer

# Load the pre-trained tokenizer associated with the specific model
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Tokenize text: yields input_ids (token IDs) and attention_mask
encoded_input = tokenizer("Transformers are highly efficient.")
```

Note: The tokenizer maps words to sub-word tokens (e.g., WordPiece), assigns numerical IDs, and automatically generates the `attention_mask` to tell the model which tokens are real data and which are padding.

Text Classification and Sentence Similarity

Because BERT processes text bidirectionally, it generates rich contextual embeddings for the entire sentence.

Text Classification: A common workflow is adding a classification head (a dense layer) on top of BERT's `[CLS]` token output to classify sequences (e.g., sentiment analysis, spam detection).

Sentence Similarity: By extracting the embeddings of two different sentences, we can compute metrics like **Cosine Similarity** to determine how semantically close they are, which is the foundation of semantic search.

Generative AI in Practice

Language Generation with GPT-Based Models

Unlike BERT, GPT (Generative Pre-trained Transformer) models use a **decoder-only** architecture optimized for auto-regressive text generation.

Remark 77 (Prompting and Generation)**.** In practical applications, GPT models are initialized with a "prompt". The model calculates the probability distribution of the next token and samples from it. Controlling parameters like **Temperature** (randomness of selection) and **Top-k / Top-p** (limiting the pool of probable next words) is essential for guiding the creativity and coherence of the generated text.

Image Generation with GANs

While Transformers dominate text, Generative Adversarial Networks (GANs) and Diffusion models are the practical standards for image generation.

- **Implementation Focus:** In a practical workflow, implementing a GAN requires setting up two competing networks—the Generator and the Discriminator.
- **Training Loop:** The training loop is notoriously delicate. You must alternate between training the Discriminator to correctly identify real vs. generated images, and training the Generator to produce images that fool the Discriminator. If one network outpaces the other, the model suffers from *mode collapse* or fails to converge.

Summary of Modern ML Workflows

Whether using BERT for classification or GPT for generation, the modern NLP pipeline follows a standardized flow:

1. **Acquire Data:** Stream datasets using `datasets`.
2. **Preprocess:** Tokenize text ensuring sequence padding and truncation.
3. **Load Model:** Initialize a pre-trained base model (Transfer Learning).
4. **Fine-tune (Optional):** Train the model on your specific dataset using an optimizer like AdamW.
5. **Evaluate:** Test the model using domain-specific metrics (e.g., Accuracy, F1 for classification; BLEU, ROUGE for generation).